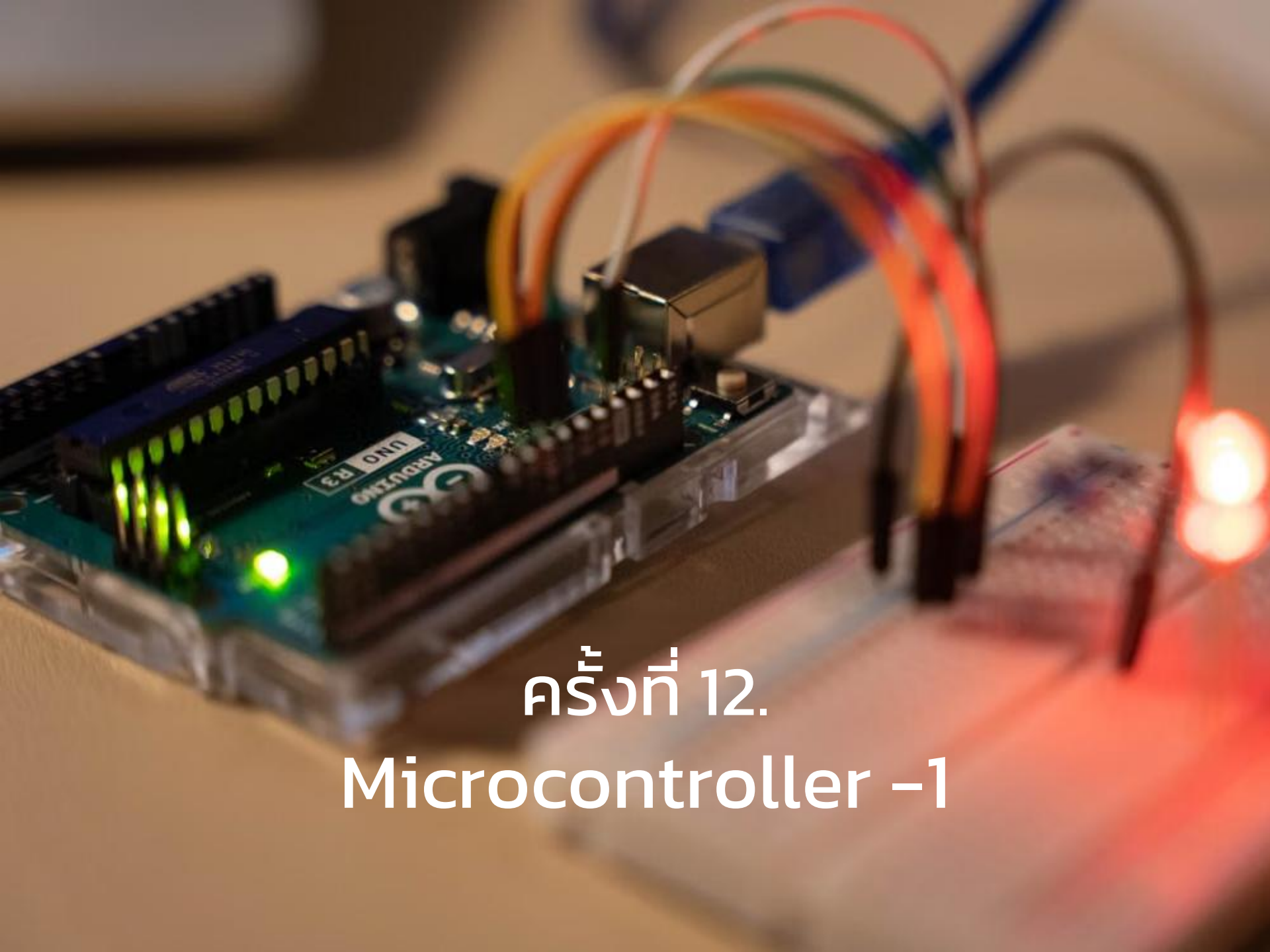


ครั้งที่ 12.
Microcontroller -1

1. Introduction

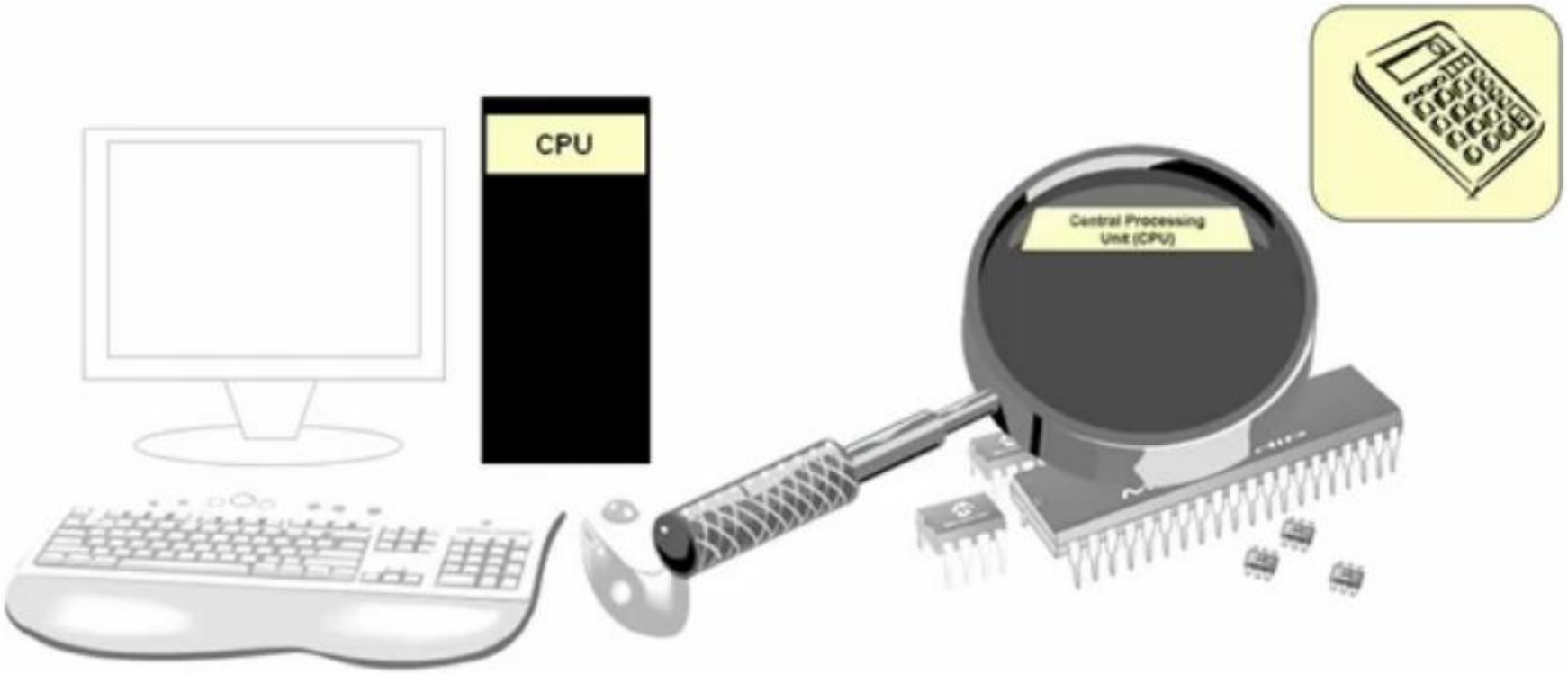


1. What is a Microcontroller?



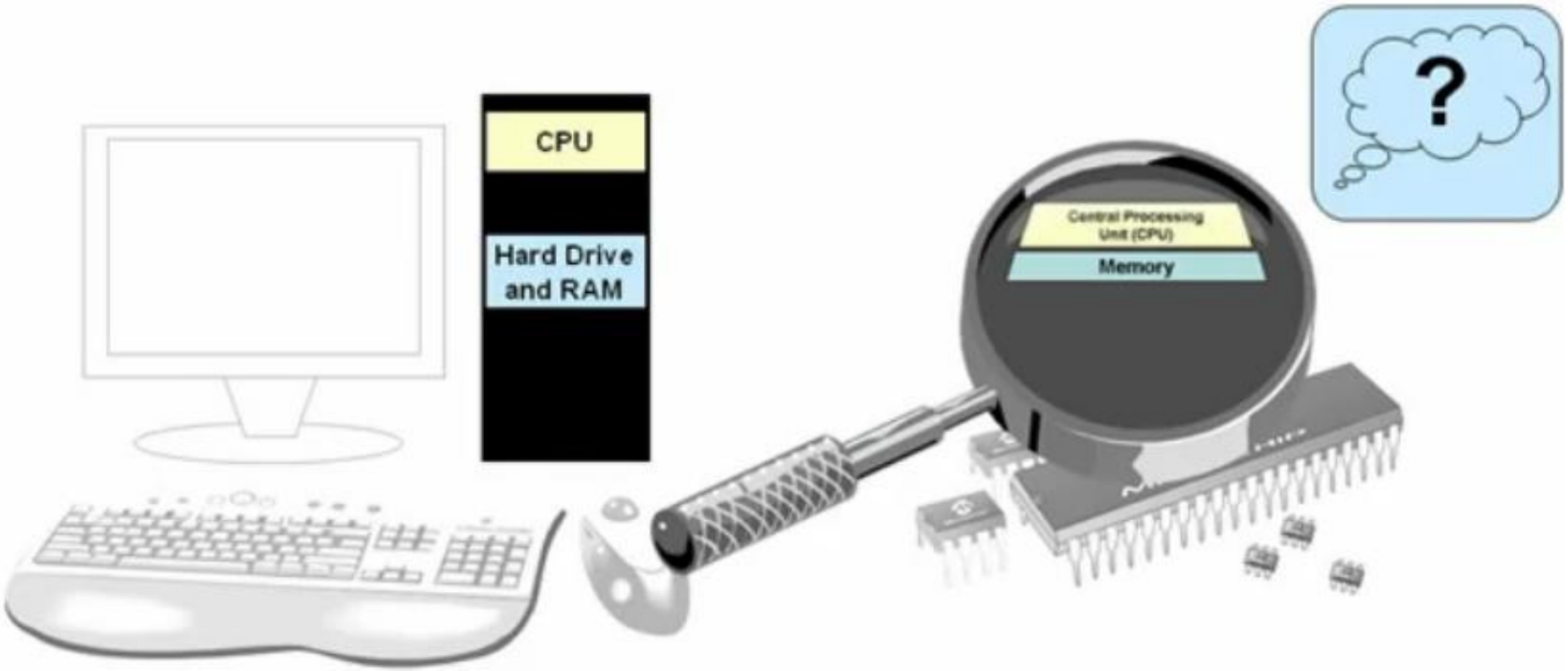
- A microcontroller (**µC**, **uC** or **MCU**) is a single integrated circuit that contains the four major parts of every computer system
- A device optimized for control applications

What is in a Microcontroller?



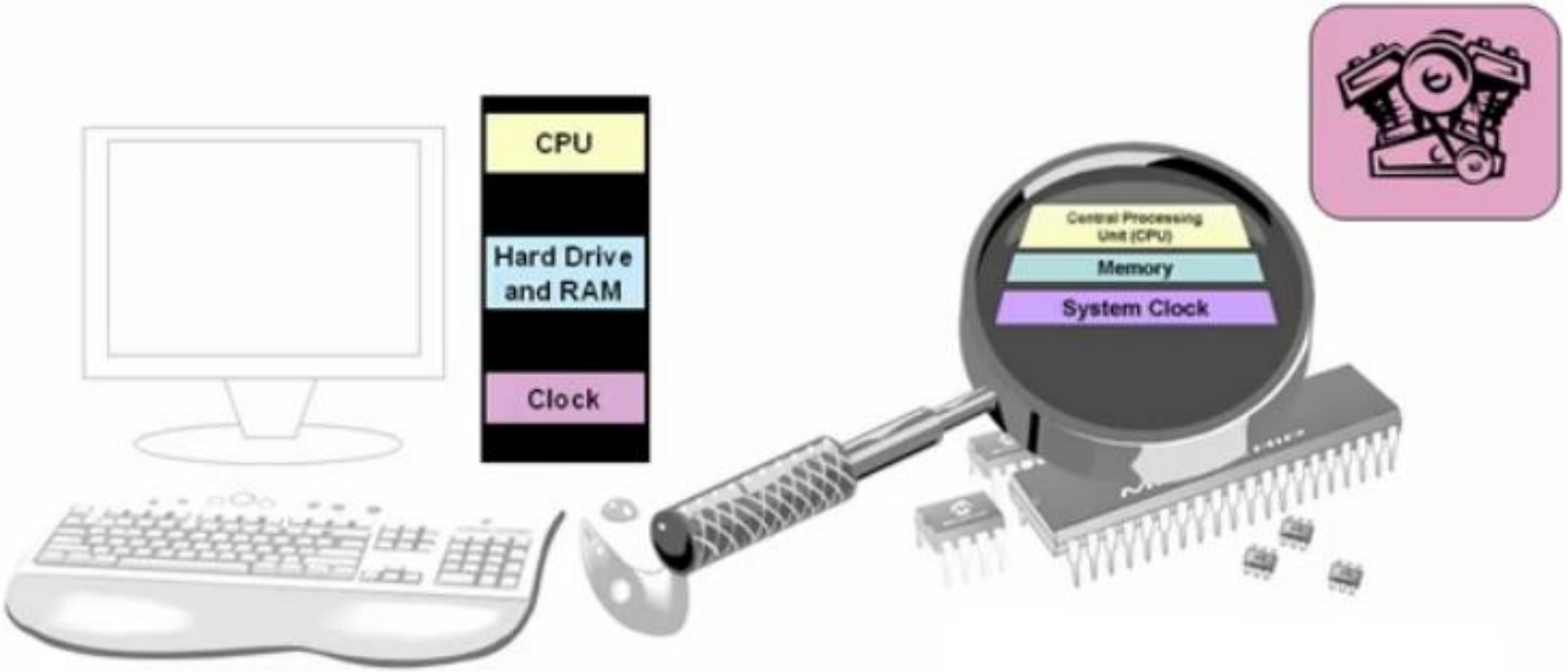
- All computers have a CPU (central processing unit) that executes programs

What is in a Microcontroller?



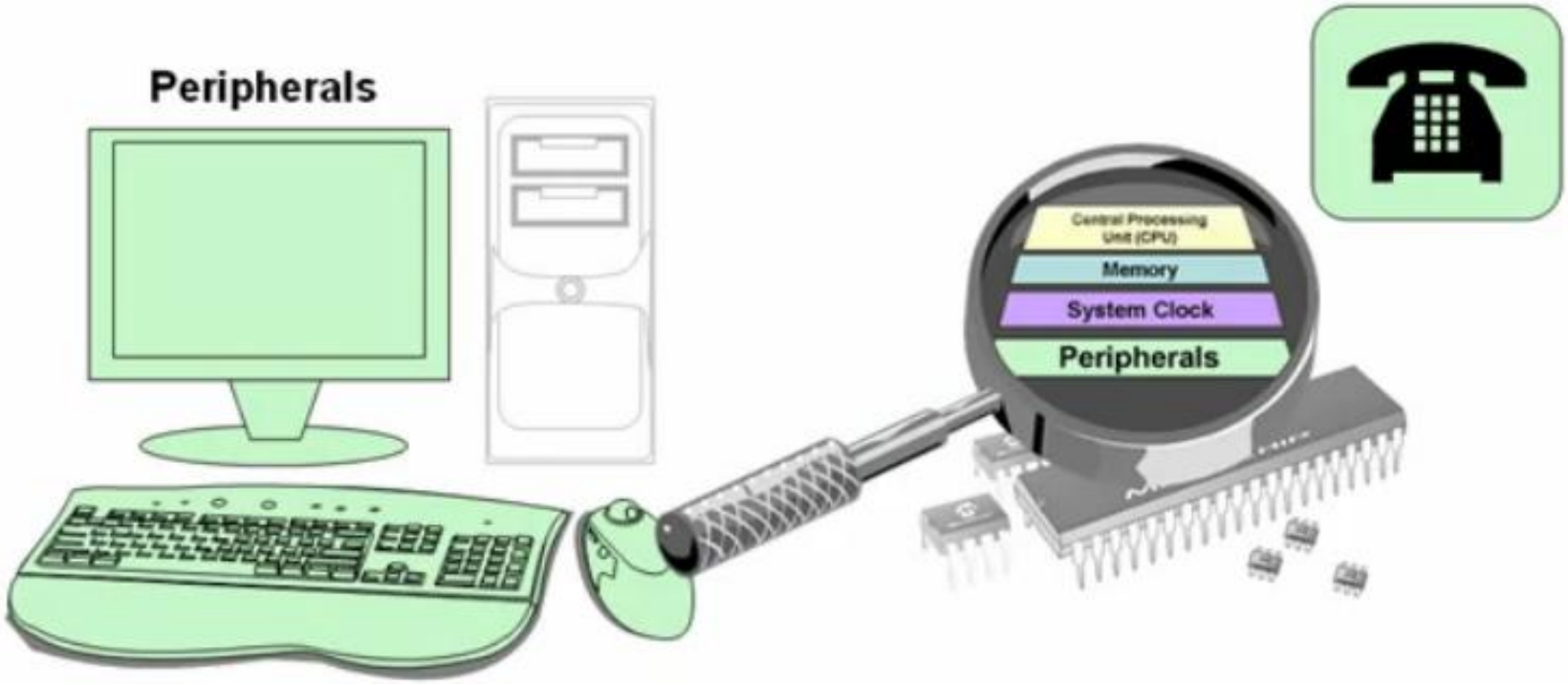
- A computer has some type of memory where it can store variables and instructions or steps
- Memory may be RAM, ROM, (E)EPROM, Flash

What is in a Microcontroller?



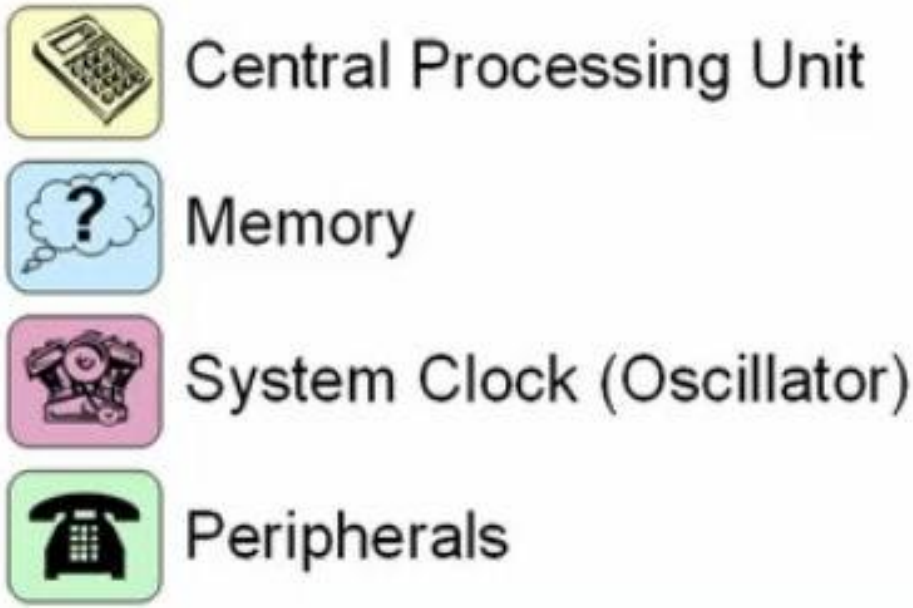
- Computers have a clock or oscillator that determine the speed of program execution

What is in a Microcontroller?



- Computers have various input and output (I/O) capabilities to support various peripherals.

What is in a Microcontroller?





- Microcontroller range from 8 – 40+ pin packages
- Extra pins provide analog and digital I/O options
- 8, 16, 32 & 64 bits instructions & data controllers

What is a Microcontroller

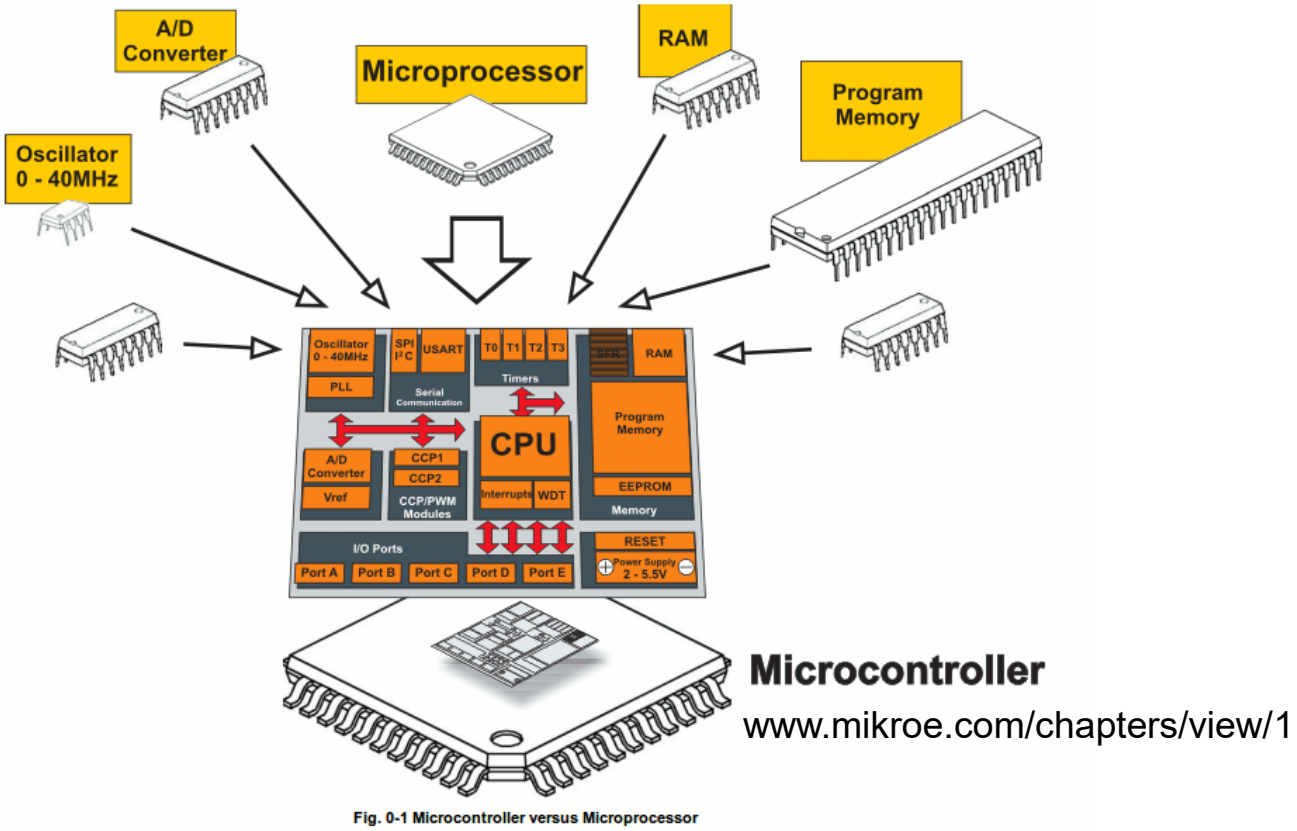
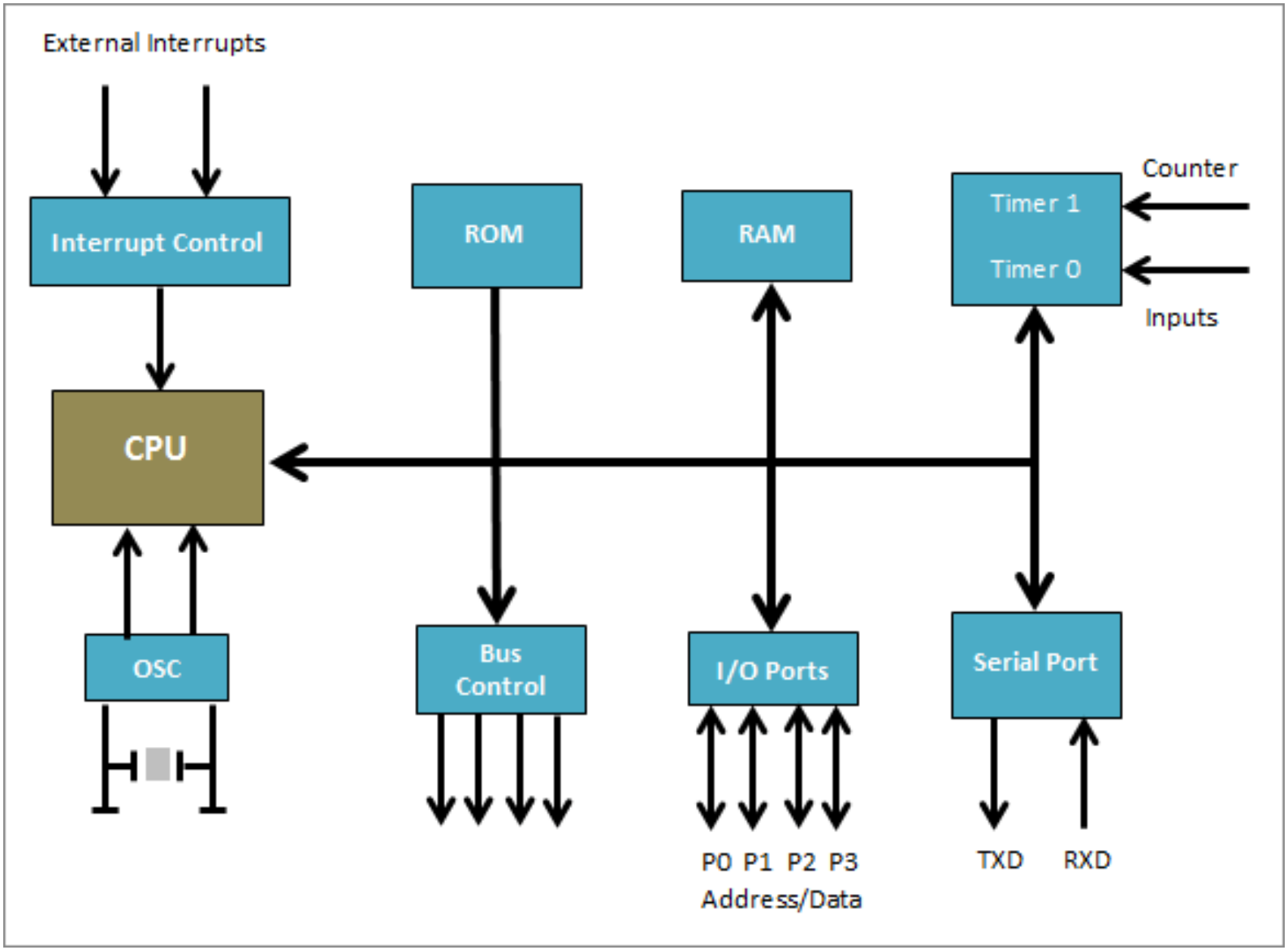


Fig. 0-1 Microcontroller versus Microprocessor

- A small computer on a single chip
  - containing a processor, memory, and input/output
- Typically, "**embedded**" inside some device that they control
- A microcontroller is often small and low cost

Block diagram of a microcontroller



Where To Find Microcontrollers?



- Microcontroller are found everywhere!
- uC’s are considered embedded systems

2. สถาปัตยกรรมของไมโครคอนโทรลเลอร์

- MCS-51 (8-bit) ออกแบบโดย intel
- MCS-96 (16-bit) ออกแบบโดย intel
- PIC ออกแบบโดย Microchip Technology
- AVR ออกแบบโดย Atmel
- ARM ออกแบบโดย ARM Holdings
- 68HC11 ออกแบบโดย Motorola
- Rabbit 2000 ออกแบบโดย Rabbit Semiconductor











3. สถาปัตยกรรมของ AVR

- แบ่งออกเป็น 2 ตระกูลคือ 8-bit AVR และ 32-bit AVR
- ในวิชานี้จะกล่าวถึงสถาปัตยกรรมของ AVR ขนาด 8 บิตเท่านั้น
- สถาปัตยกรรม AVR ออกแบบโดย ATMEL เมื่อปี 1996 เป็นซีพียูแบบ **RISC** (Reduced Instruction Set Computer)
- มีสถาปัตยกรรมการต่อหน่วยความจำแบบ Harvard ซึ่งแยกหน่วยความจำโปรแกรมและหน่วยความจำข้อมูล ออกจากกันโดยเด็ดขาด
- ใช้หน่วยความจำแบบ Flash สำหรับเป็นหน่วยความจำโปรแกรม
- ใช้หน่วยความจำแบบ SRAM สำหรับหน่วยความจำข้อมูล
- นอกจากนี้ยังมีหน่วยความจำแบบ EEPROM ซึ่งสามารถเก็บข้อมูลเอาไว้ได้โดยไม่จำเป็นต้องมีไฟเลี้ยง

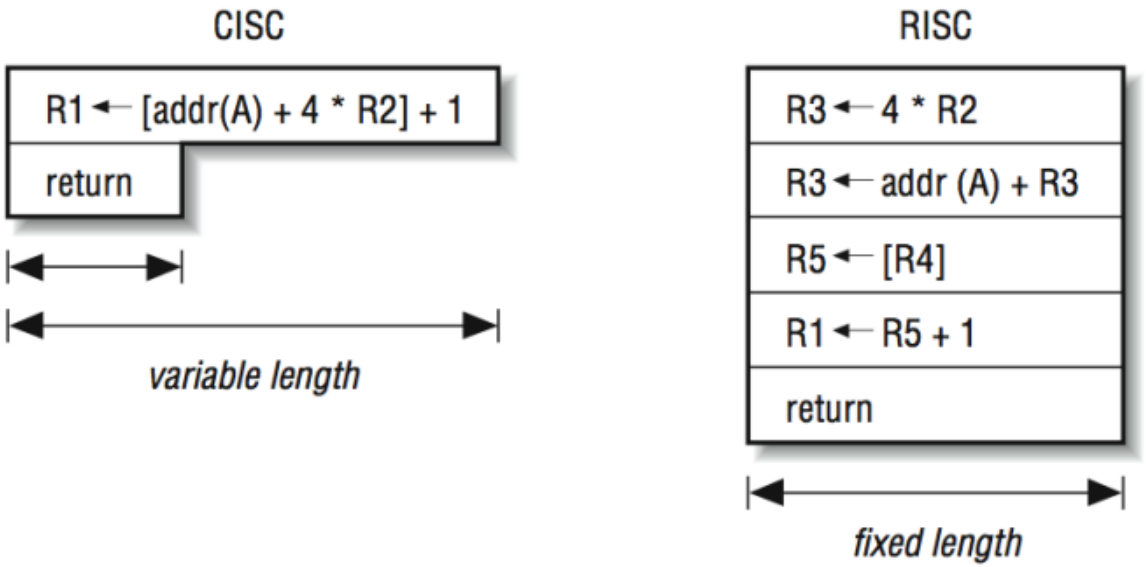
Difference between RISC and CISC Embedded Architecture



Two Specific Computing in the CPU

- Computer architecture of the CPU has two categories.
  - Reduced Instruction Set Computer (RISC)
    - Focus on **small** instruction set and running in a clock cycle. *(Simpler instructions, longer code)*
    - A long process build from a sequence of small instruction set.
    - Easy to pipeline
    - CPU: PowerPC, Atmel's AVR, Microchip PIC, ARM processors,
  - Complex Instruction Set Computer (CISC)
    - Focus on a work completed in an instruction set by using a long clock cycle. *(Longer instructions, shorter code)*
    - Hard to pipeline
    - CPU: Intel x86 architectures
  - Hybrid RISC/CISC
    - AMD processors

Compare ADD instruction



- Suppose the code below comparing

CISC

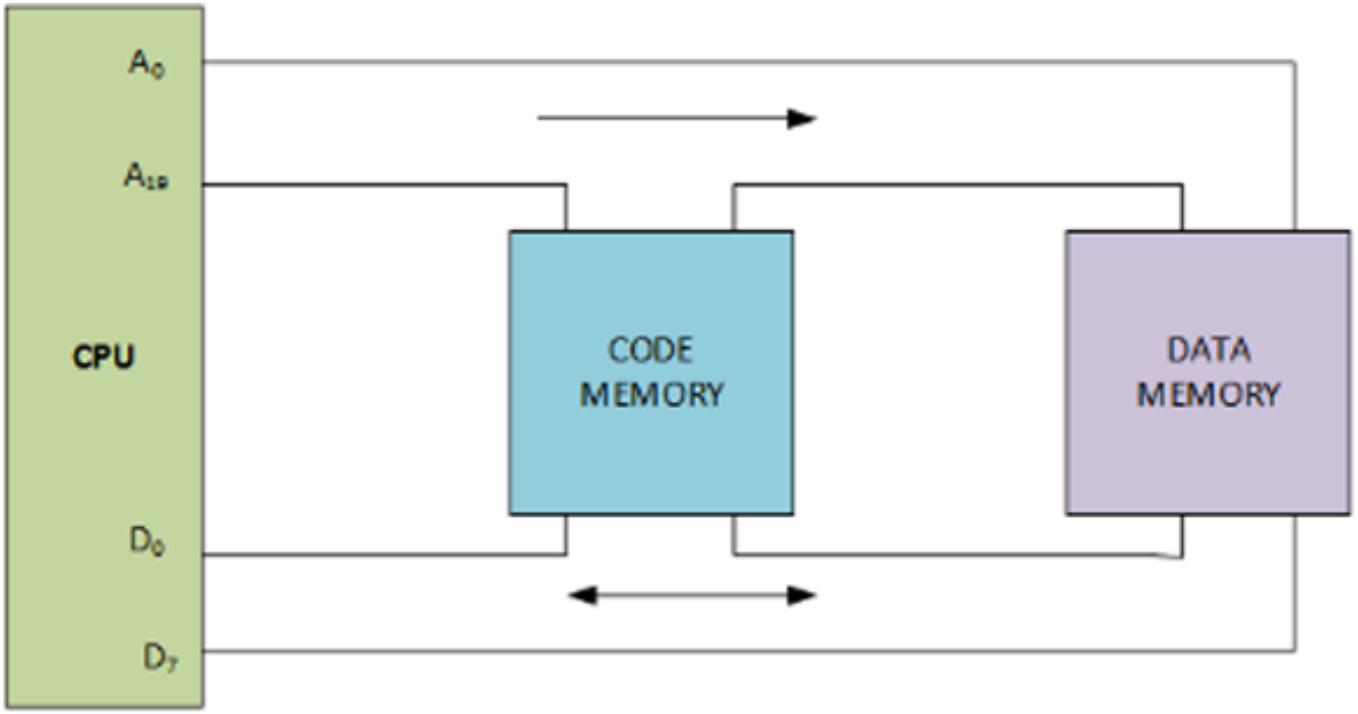
```
mov  ax, 10   ← 1
mov  bx, 5    ← 1
mul  bx, ax   ← 30
```

RISC

```
        mov  ax, 0    ← 1
        mov  bx, 10   ← 1
        mov  cx, 5    ← 1
begin:  add  ax, bx   ← 1
        loop cx, begin ← 1
```

Princeton Architecture & Harvard architecture

Every CPU has a memory associated with it to store the program and data. Earlier a single bus was used for getting access to the program and data. A single bus for getting the code and data means, they come to get in each other's way and **slow down the processing speed of the CPU** because each had to wait for the other to finish the fetching.

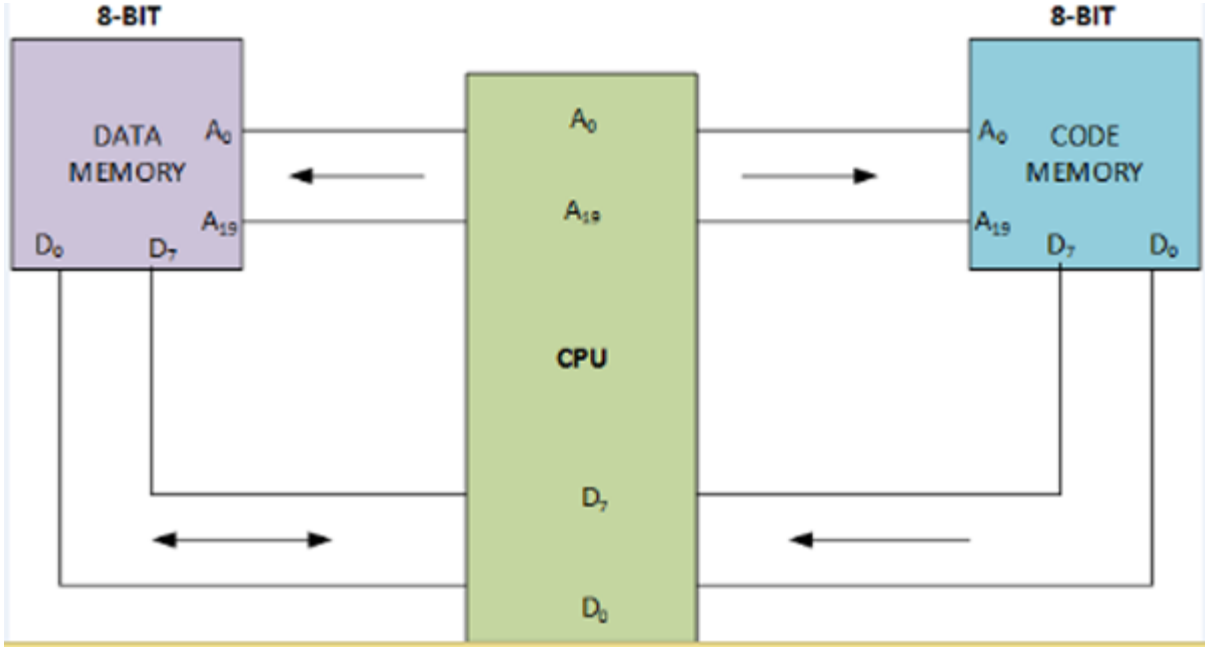


Princeton architecture

Princeton Architecture & Harvard architecture

To speed up the process **Harvard Architecture** was proposed. In this architecture a separate data buses for data and program are present. So it means this architecture proposed the use of four buses

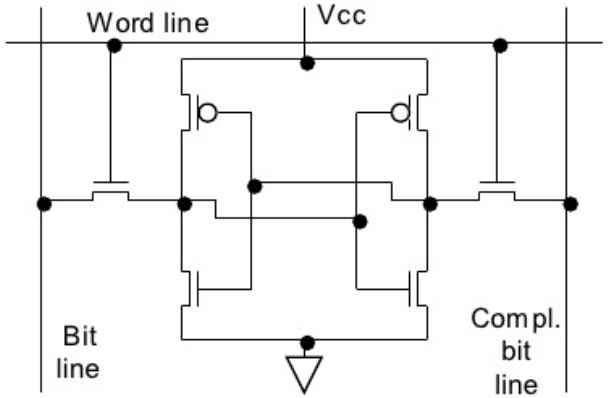
1. A set of data bus carrying the data into and out of the CPU.
2. A set of address bus for accessing the data.
3. A set of data bus for carrying code into the CPU.
4. An address bus for accessing the code.



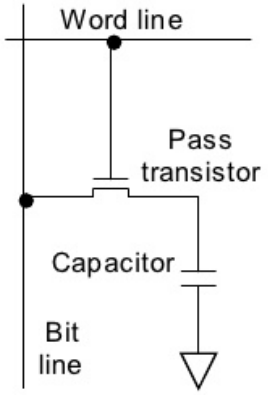
Harvard Architecture

SRAM & DRAM

Static RAM	Dynamic RAM
➢ SRAM uses transistor to store a single bit of data	➢ DRAM uses a separate capacitor to store each bit of data
➢ SRAM does not need periodic refreshment to maintain data	➢ DRAM needs periodic refreshment to maintain the charge in the capacitors for data
➢ SRAM's structure is complex than DRAM	➢ DRAM's structure is simplex than SRAM
➢ SRAM are expensive as compared to DRAM	➢ DRAM's are less expensive as compared to SRAM
➢ SRAM are faster than DRAM	➢ DRAM's are slower than SRAM
➢ SRAM are used in Cache memory	➢ DRAM are used in Main memory



(b) Typical SRAM cell



(a) DRAM cell

4. ประเภทการใช้งาน AVR ขนาด 8 บิต

- tinyAVR เป็นซีพียูในรุ่นเล็ก ซึ่งต้องการความเล็กกะทัดรัดของวงจร โดยเหมาะกับระบบควบคุมขนาดเล็กๆ ที่ต้องการหน่วยความจำและวงจรสนับสนุนไม่มากนัก ซีพียูในรุ่นนี้จะมีราคาถูกกว่ากลุ่มอื่น



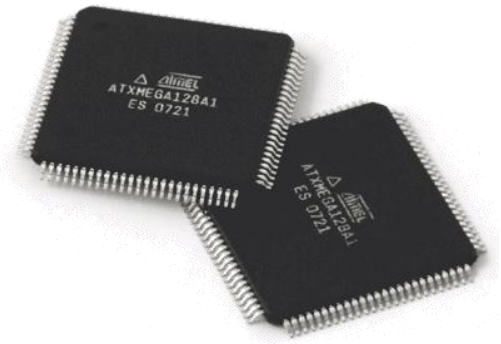


- megaAVR จะมีชื่ออีกอย่างว่า ATmega โดยมีวงจรสนับสนุนภายในเพิ่มเติมตลอดจนเพิ่มขนาดของหน่วยความจำให้ใช้งานมากกว่าตระกูล Tiny เหมาะกับงานควบคุมทั่วๆ ไป





- XMEGA เพิ่มความละเอียดของวงจร A/D จากปกติมีความละเอียด 10 บิตในรุ่นเล็กกว่าเป็น 12 บิต และวงจร DMA controller ซึ่งช่วยลดภาระของซีพียูในการควบคุมการรับส่งข้อมูลระหว่างอุปกรณ์ I/O กับหน่วยความจำ



- FPSLIC (AVR core with FPGA) สำหรับงานที่ต้องการควบคุมที่ต้องการความยืดหยุ่นในขั้นตอนการออกแบบและพัฒนา โดยผู้ออกแบบสามารถออกแบบวงจรในระดับฮาร์ดแวร์เพิ่มเติมด้วยภาษาบรรยายฮาร์ดแวร์ (HDL: Hardware Description Language) เช่น ภาษา VHDLหรือภาษา Verilog และให้วงจรที่ออกแบบทำงานร่วมกับซีพียู AVR core

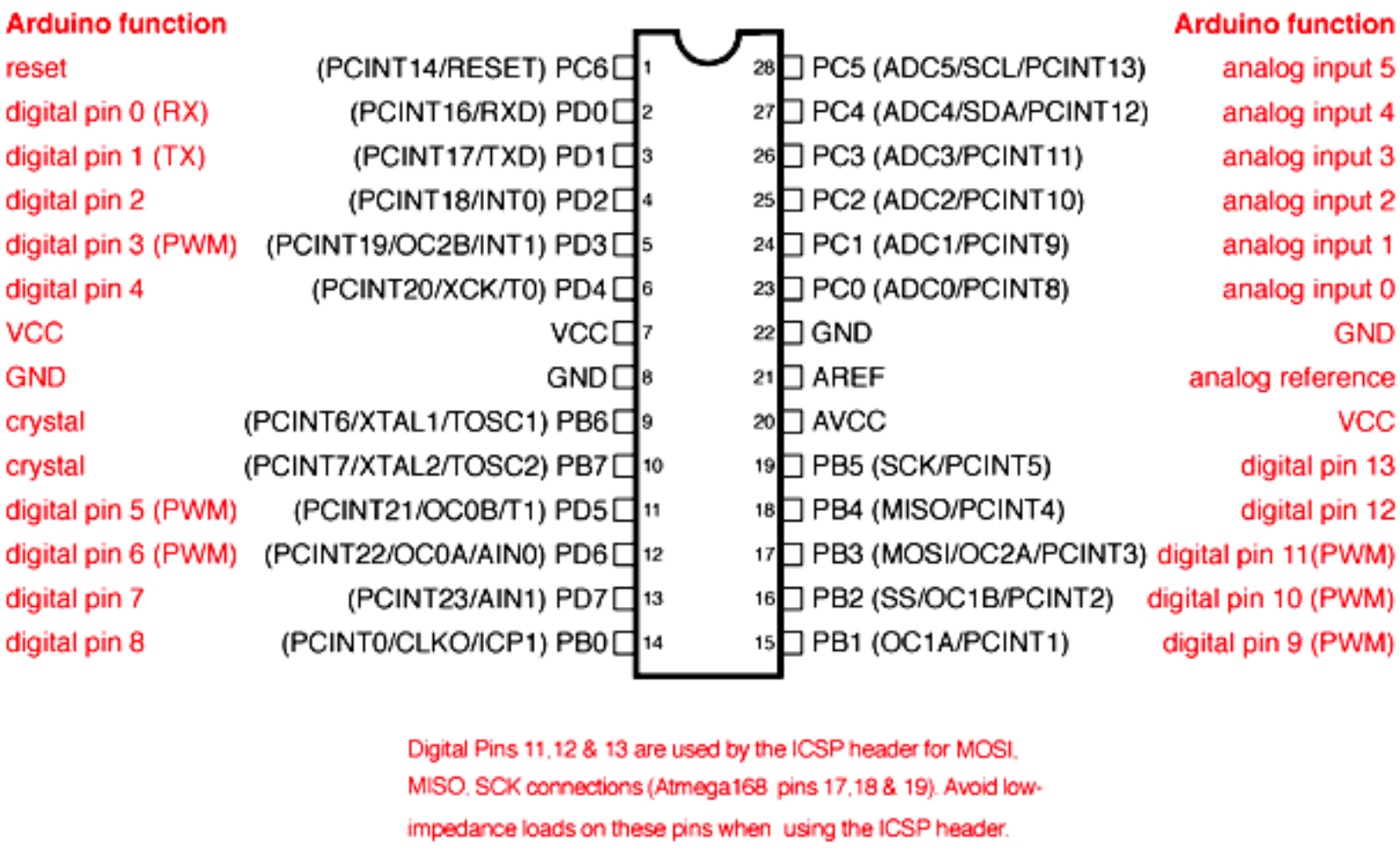
- Application Specific AVR เป็นซีพียูที่ออกแบบมาโดยเพิ่มวงจรควบคุมเฉพาะด้านเข้าไปซึ่งไม่ พบในซีพียูกลุ่มอื่นๆ เช่นวงจร USB controller หรือวงจร CAN bus เป็นต้น

AVR มีให้เลือกใช้งานหลายเบอร์ แต่ละเบอร์จะมีขนาด ราคา ความสามารถ และขนาด หน่วยความจำตลอดจนถึงวงจรสนับสนุนภายในที่แตกต่างกันออกไป

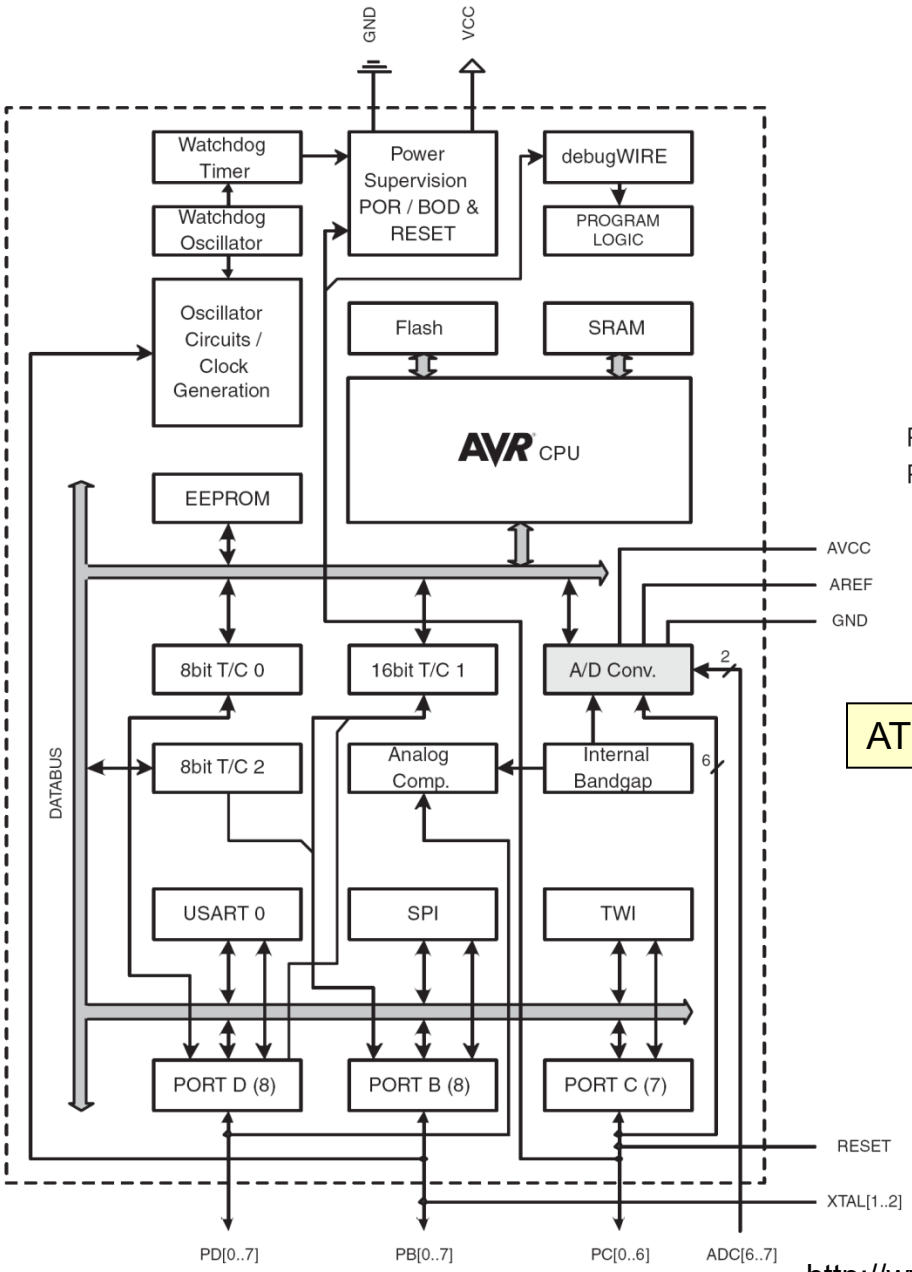
5.1 ATmega328

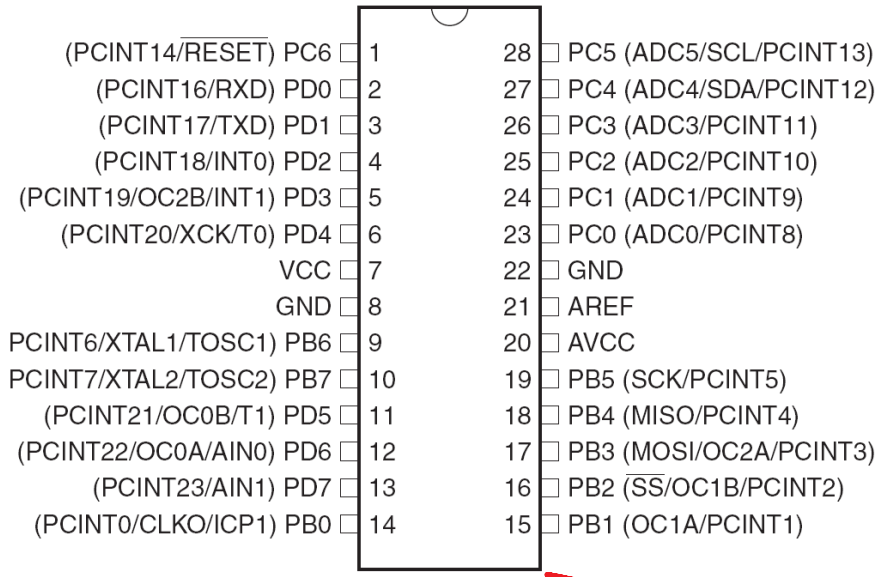
- หน่วยความจำโปรแกรมแบบ FLASH ขนาด 32 Kbyte
- หน่วยความจำข้อมูลแบบ SRAM ขนาด 2 Kbyte
- หน่วยความจำข้อมูลแบบ EEPROM ขนาด 1 Kbyte
- สนับสนุนการเชื่อมต่อแบบ I^2C bus , SPI , UART
- พอร์ตอินพุตเอาต์พุตจำนวน 20 ports
- Analog input (ADC) 6 Port ขนาด 10 bit
- วงจรนับ/จับเวลาขนาด 8 บิต จำนวน 2 ตัว และขนาด 16 บิตจำนวน 1 ตัว
- สนับสนุนช่องสัญญาณสำหรับสร้าง PWM จำนวน 6 ช่องสัญญาณ
- ทำงานได้ตั้งแต่ย่านแรงดัน 1.8-5.5 Volts
- ความถี่ใช้งานสูงสุด 20 MHz

รายละเอียด ขา ต่างๆ ของ ATmega328



ATmega328 Internal Architecture





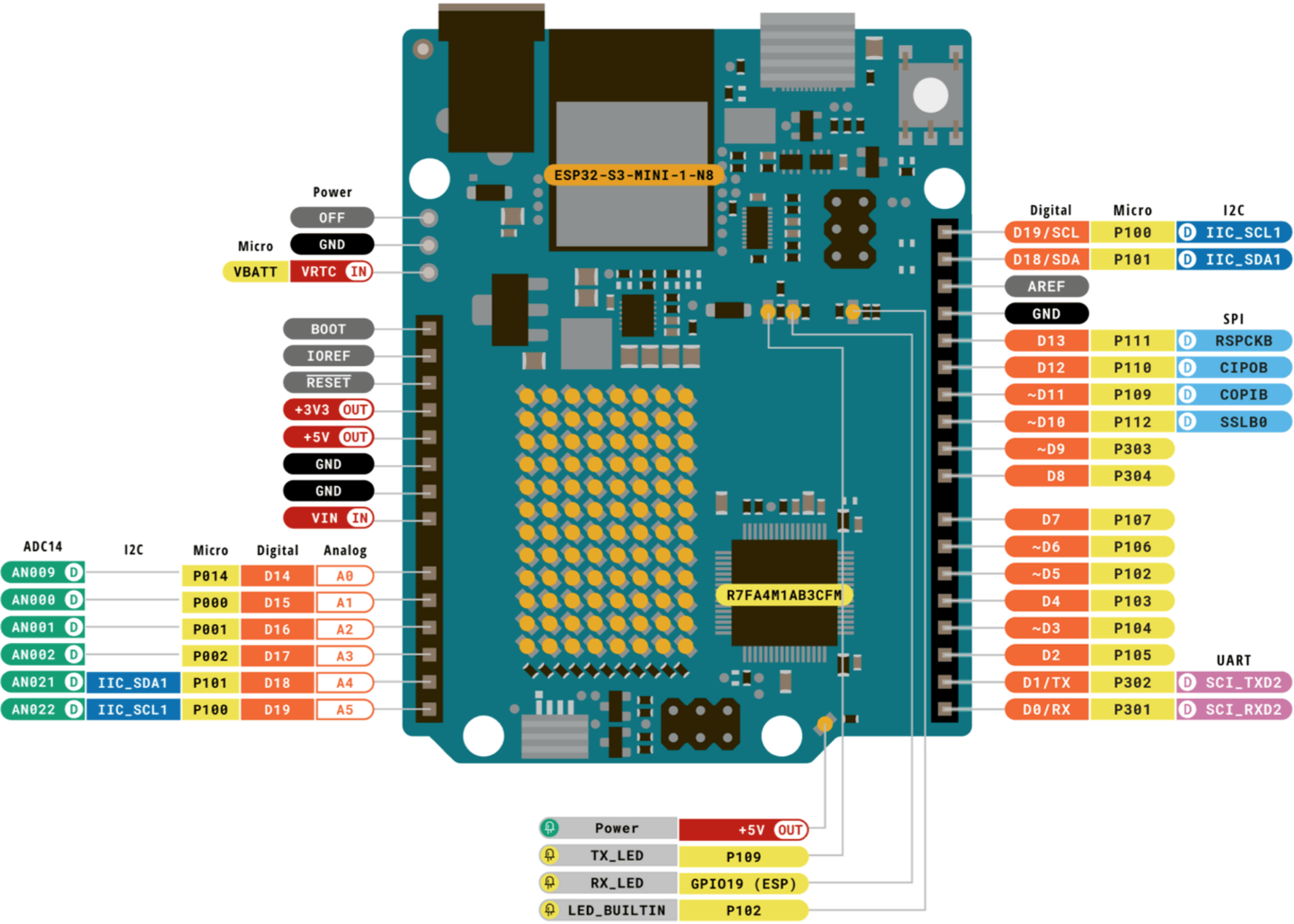
ATmega328 data sheet pp. 2, 5



http://www.adafruit.com/index.php?main_page=popup_image&pID=50

5.2 Renesas RA4M1 (Arm Cortex-M4F)

- หน่วยความจำโปรแกรมแบบ FLASH ขนาด 256 Kbyte
- หน่วยความจำข้อมูลแบบ SRAM ขนาด 32 Kbyte
- หน่วยความจำข้อมูลแบบ EEPROM ขนาด 8 Kbyte
- สนับสนุนการเชื่อมต่อแบบ I^2C bus , SPI, UART, CAN
- พอร์ตอินพุตเอาต์พุตจำนวน 20 ports
- Analog input (ADC) 6 port ขนาด 14 bit, DAC 1 port (12 bit)
- วงจรนับ/จับเวลาขนาด 8 บิต จำนวน 2 ตัว และขนาด 16 บิตจำนวน 1 ตัว
- สนับสนุนช่องสัญญาณสำหรับสร้าง PWM จำนวน 6 ช่องสัญญาณ
- ทำงานที่แรงดัน 5 Volts
- ความถี่ใช้งานสูงสุด 48 MHz
- WiFi 802.11 b/g/n 2.4GHz band , Bluetooth 5.0


ESP32-S3-MINI-1-N8
R7FA4M1AB3CFM
Power
OFF
Micro
GND
VBATT
VRTC IN
BOOT
IOREF
RESET
+3V3 OUT
+5V OUT
GND
GND
VIN IN
ADC14
I2C
Micro
Digital
Analog
AN009 D
AN000 D
AN001 D
AN002 D
AN021 D
AN022 D
IIC_SDA1
IIC_SCL1
P014
P000
P001
P002
P101
P100
D14
D15
D16
D17
D18
D19
A0
A1
A2
A3
A4
A5
Digital
Micro
I2C
D19/SCL
D18/SDA
P100
P101
D IIC_SCL1
D IIC_SDA1
AREF
GND
SPI
D13
D12
~D11
~D10
~D9
D8
P111
P110
P109
P112
P303
P304
D RSPCKB
D CIPOB
D COPIB
D SSLB0
D7
~D6
~D5
D4
~D3
D2
P107
P106
P102
P103
P104
P105
UART
D1/TX
D0/RX
P302
P301
D SCI_TXD2
D SCI_RXD2
Power
+5V OUT
TX_LED
P109
RX_LED
GPIO19 (ESP)
LED_BUILTIN
P102

6. What is Arduino ? [1]

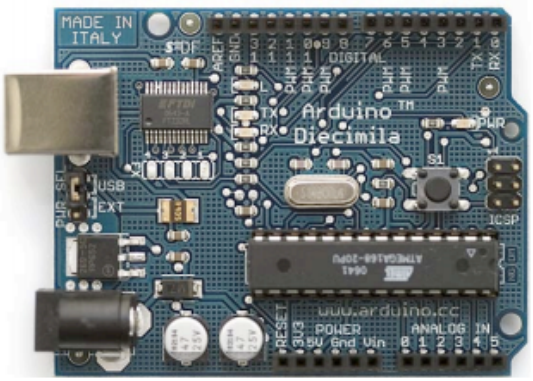
- Arduino เป็นโครงการพัฒนาไมโครคอนโทรลเลอร์ของตระกูล AVR แบบ Open Source
- Arduino เป็นชื่อเรียกของ platform ซึ่งประกอบไปด้วย
  - Development Board ที่ใช้ microcontroller ตระกูล AVR
  - IDE เพื่อใช้ในการพัฒนาโปรแกรม โดย IDE พัฒนาโดยภาษา Java ส่งผลให้สามารถทำงานได้ทุก OS เช่น Linux, MAC OS, Windows

    และ IDE มี library มาตรฐาน ในการอ้างอิงกับบอร์ด Arduino จำนวนมาก ทำให้สะดวกในการพัฒนาโปรแกรม
- โดย microcontroller ตระกูล AVR ที่เอามาใช้นั้น จะต้องมีการติดตั้ง Firmware ไว้แล้ว โดยหน้าที่หลักของ Firmware คือ การรับโปรแกรมที่ Compile แล้ว จาก IDE มาเขียนไว้ที่ตัวมันเอง เพื่อทำงานตามโปรแกรมที่เขียน

What is Arduino ? [2]

The word "Arduino" can mean 3 things

A physical piece of hardware



A programming environment

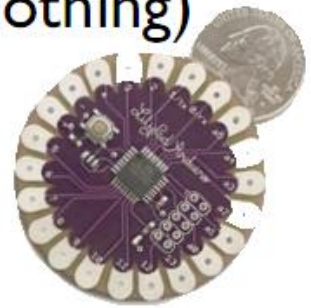


A community & philosophy

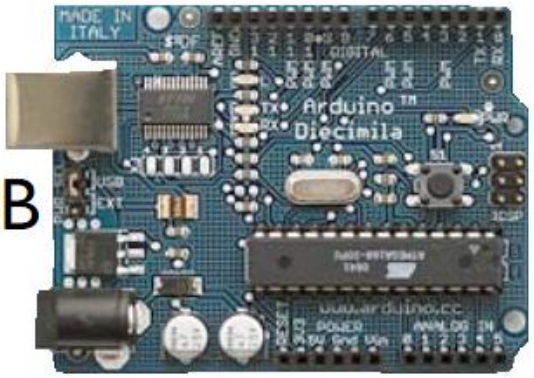


6.1 Arduino Hardware Variety

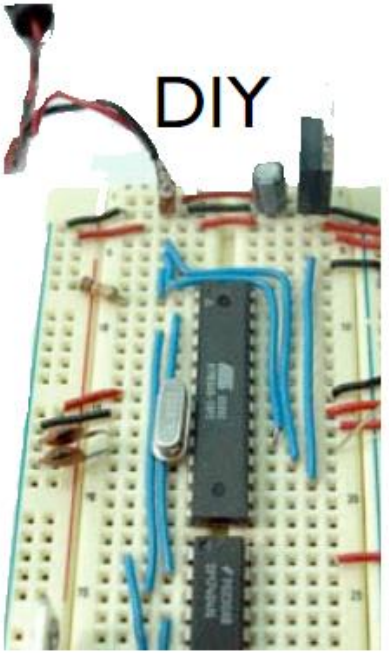
LilyPad
(for clothing)



USB



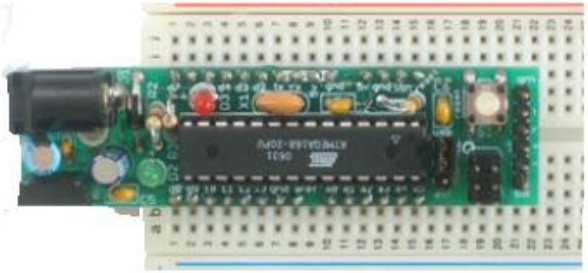
DIY





Bluetooth

Boarduino Kit



"Stamp"-sized



many different variations to suite your needs

Openness has its advantages, many different varieties.
Anyone can build an Arduino work-alike in any form-factor they want.
Product images from Sparkfun.com and Adafruit.com

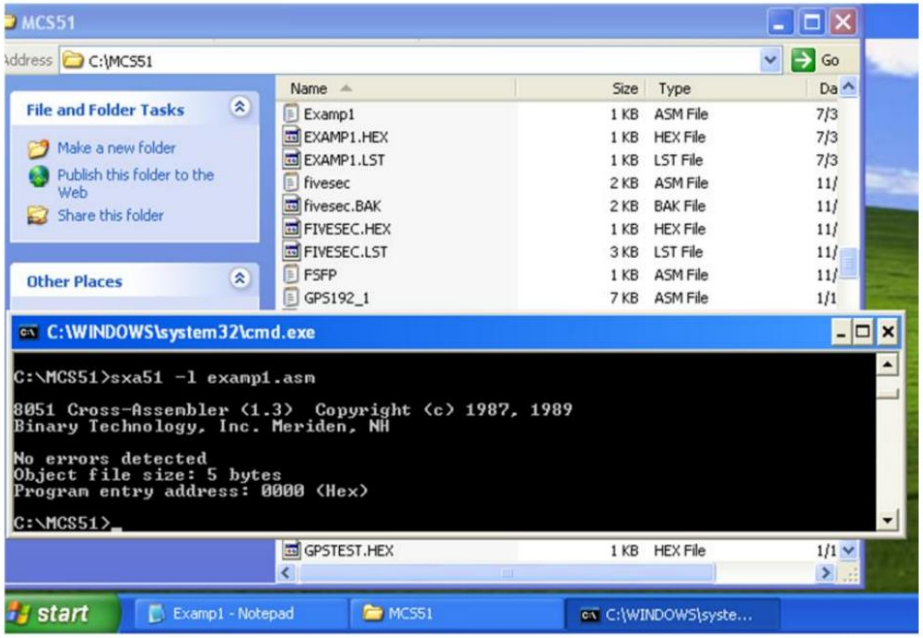
6.2 IDE (Integrated Development Environment)

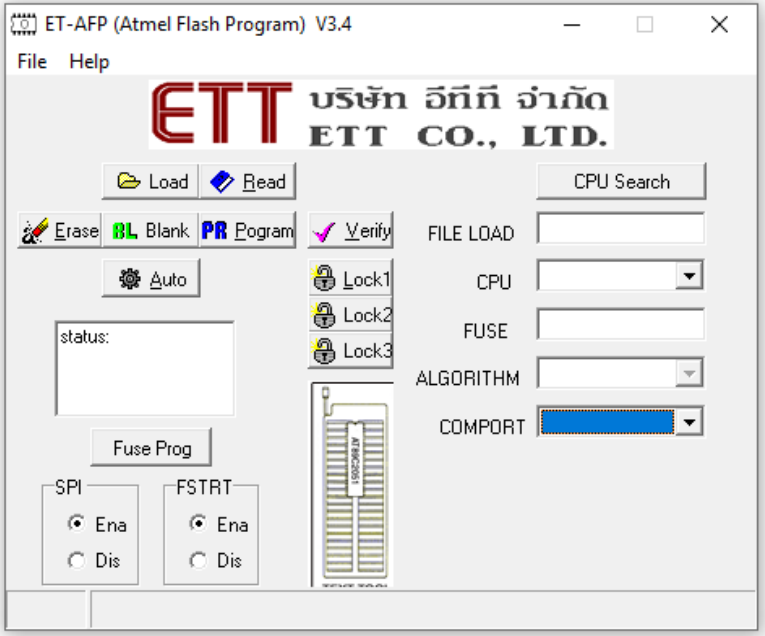
- ในอดีต การพัฒนาโปรแกรมขึ้นมา ต้องใช้โปรแกรมอื่นๆ หลายตัวในการสร้างโปรแกรม
- เช่น ต้องการเขียนโปรแกรมลงบอร์ด MCS51

  มีขั้นตอน
  - เขียนโปรแกรม โดยใช้โปรแกรม text editor เช่น notepad
  - Compile program โดยใช้โปรแกรม SXA51
  - Upload โปรแกรม โดยใช้โปรแกรม ET-AFP
  - ดูค่าที่ได้จากโปรแกรม โดยใช้โปรแกรม hyperterminal

จึงเป็นที่มาของการพัฒนา IDE ขึ้นมา โดยรวมโปรแกรมย่อยต่างๆ ให้เป็นโปรแกรมเดียว ที่มีสภาพแวดล้อมที่อำนวยต่อการพัฒนาโปรแกรมให้ง่ายขึ้น

IDE (Integrated Development Environment)

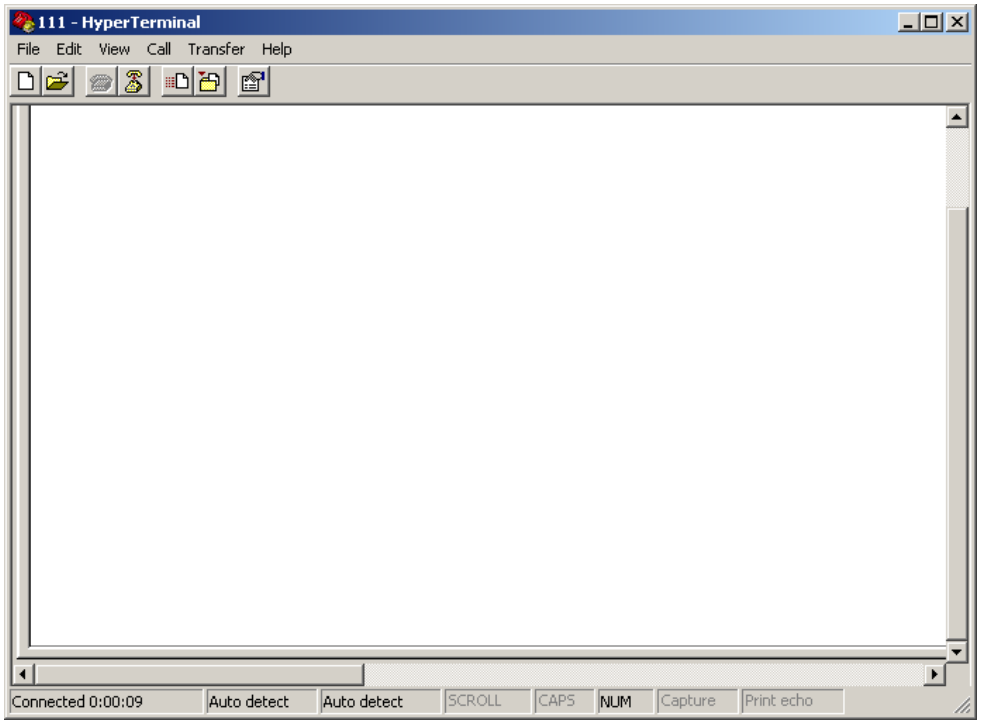


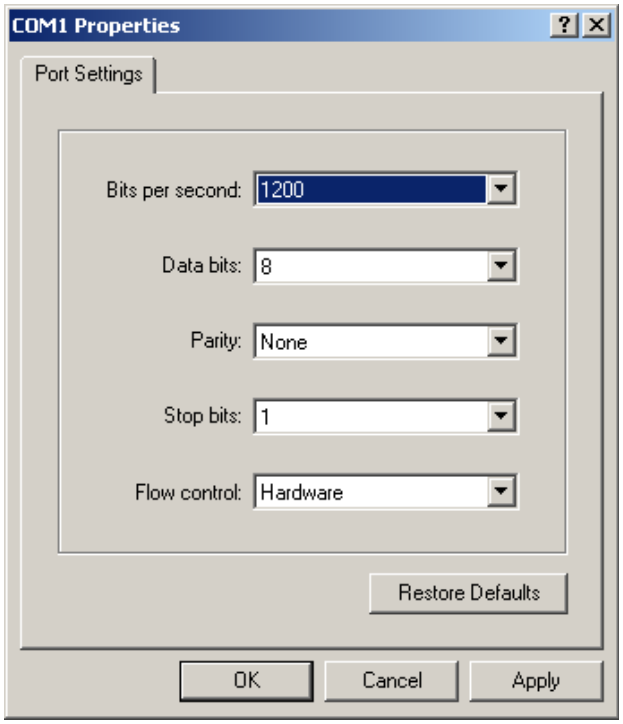


Complie file.asm to file.hex

Upload file.hex to board

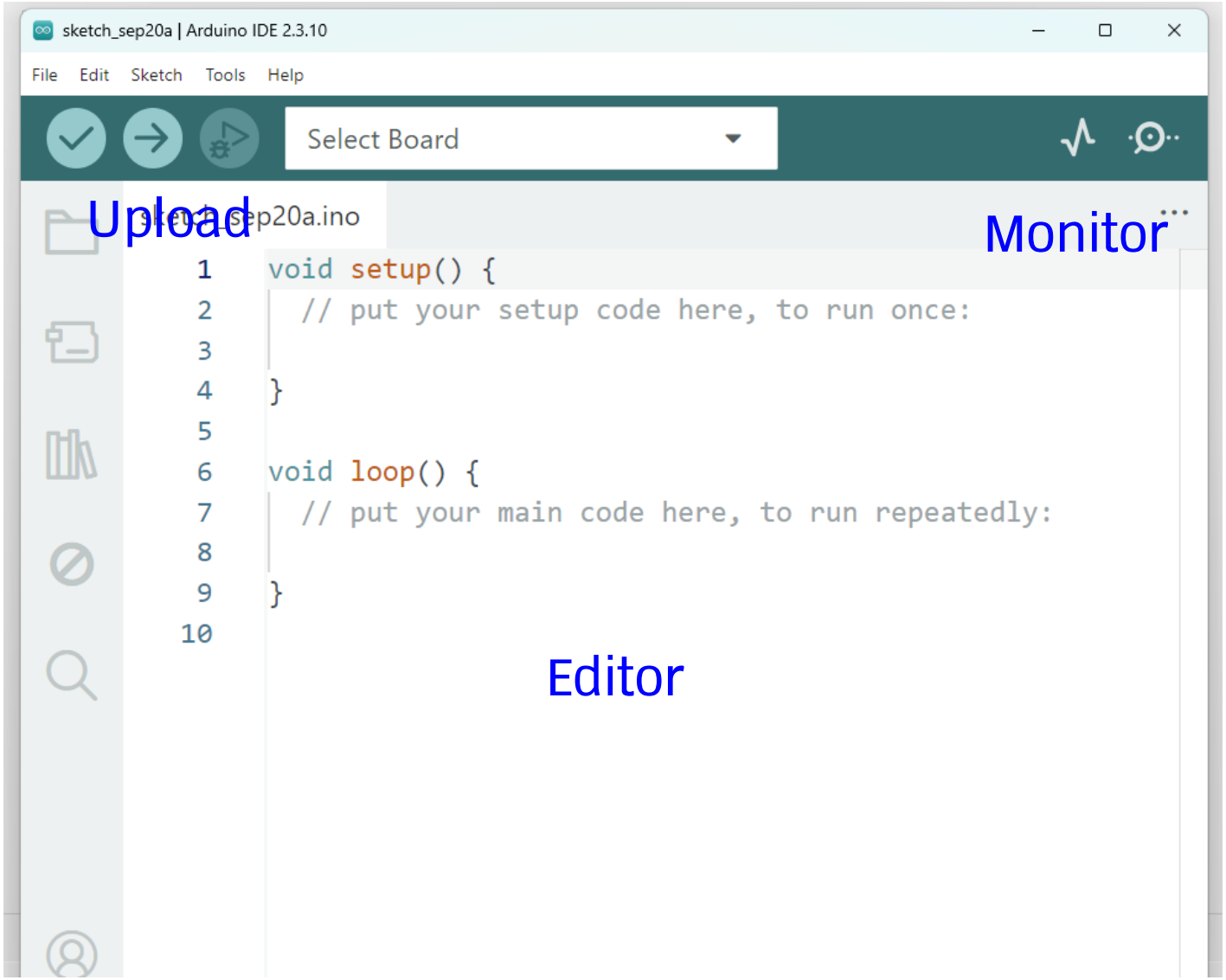
IDE (Integrated Development Environment)





หน้าจอโปรแกรม hyperterminal

6.2. Arduino IDE



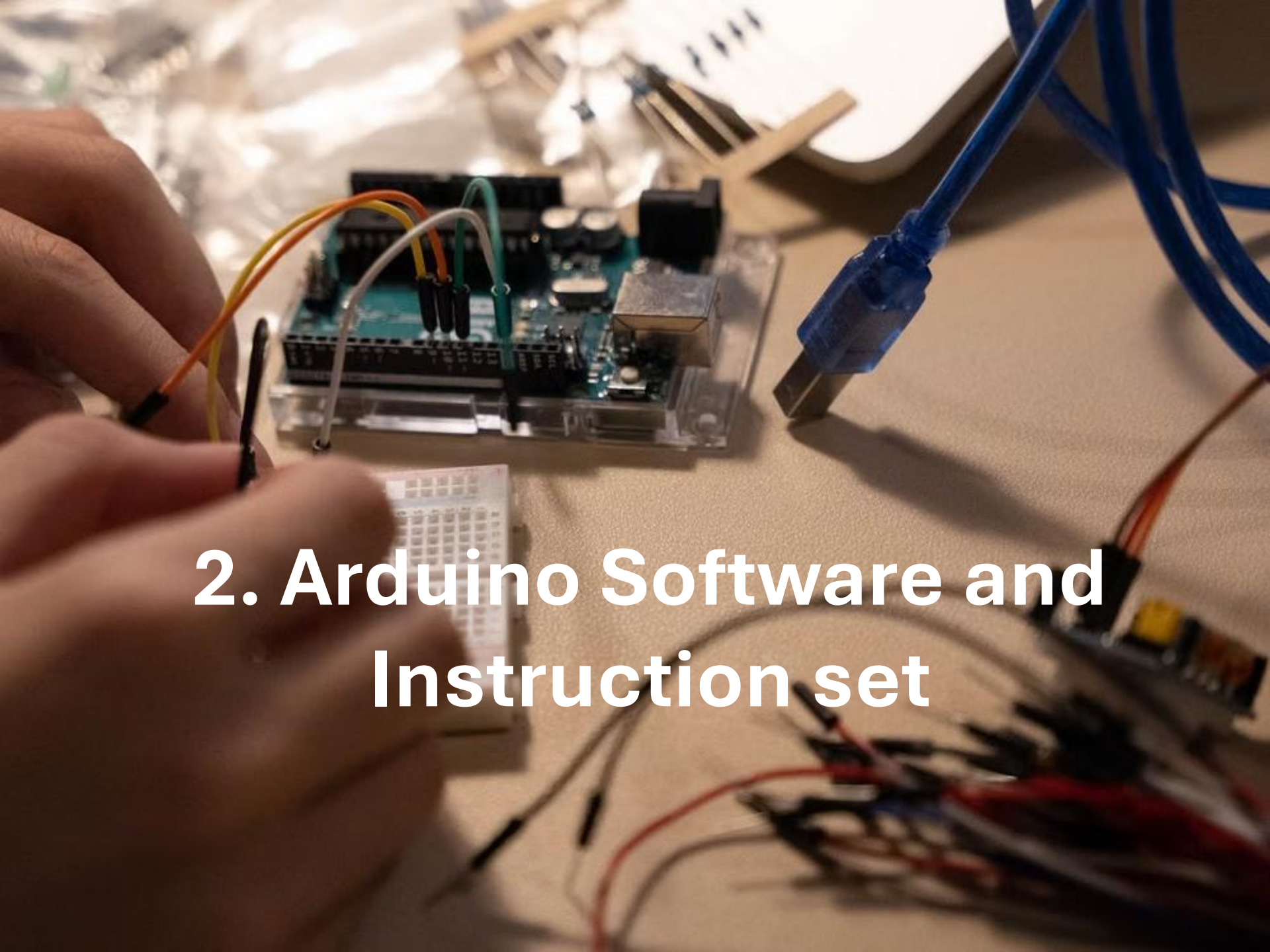
6.3 ข้อดี-ข้อเสีย Arduino

ข้อดี

- พัฒนาโปรแกรมด้วยภาษา c++ จึงสามารถใช้คุณสมบัติ OOP ได้
- ไม่ต้องใช้อุปกรณ์ ISP ในการโหลดโปรแกรมไปยัง microcontroller
- โครงสร้างการเขียนเข้าใจง่าย ไม่ซับซ้อน
- ต้นทุนมีราคาถูก
- เป็นโครงการ opensource จึงทำให้มีผู้ให้ความสนใจมาก และมี Library ให้ใช้งานเป็นจำนวนมาก

ข้อเสีย

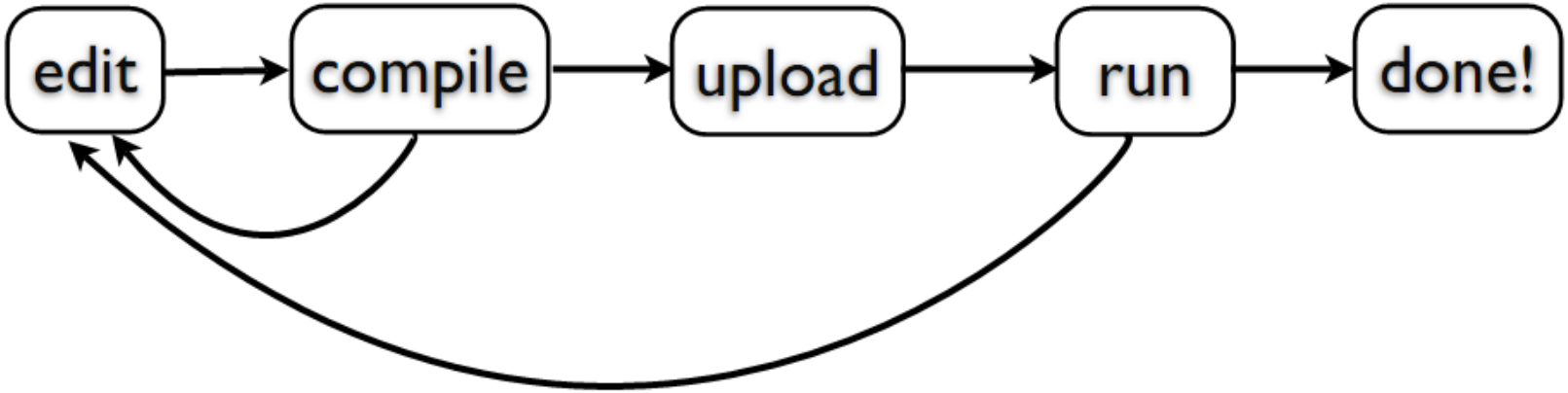
- เนื่องจากพัฒนาโปรแกรมด้วยภาษา c++ จึงใช้ทรัพยากรมากพอสมควร



2. Arduino Software and Instruction set

1. ขั้นตอนการพัฒนาโปรแกรมด้วย Arduino (Development Circle)

- Make as many changes as you want
- Not like most web programming: edit → run
- Edit → compile → upload → run

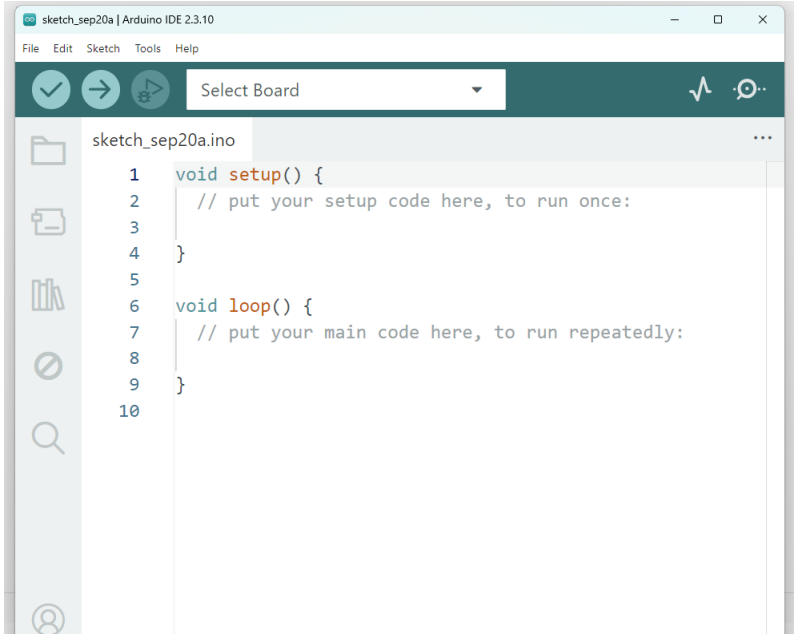


2. Arduino IDE

The Arduino programming platform was designed in JAVA to help newcomers become familiar with programming. The language used to write code is C/C++ .

Each Arduino program is called a **SKETCH** and has two required functions, called ROUTINES.

```
void setup ( ) { } ,
void loop ( ) { }
```



3. Arduino Program (Sketch) Structure

Declare variables at top

- Initialize
  - **`setup()`** – run ONCE at beginning, set pins
- Running
  - **`loop()`** – run repeatedly, after **`setup()`**

All of the code within the curly braces will be run again, and again, until the power is removed.

Example of a bare minimum program:

```
void setup()
{
}

void loop()
{
}
```

4. Programming - Syntax

Similar to C, the formatting requirement is the same.

//	- Single line comment
/* */	- Multiline comment
{ }	- used to define a block of code that starts and ends.
;	- used to define the end of a line of code.

5. Programming Concepts: Variables

ProtosnapProMiniExample2 §

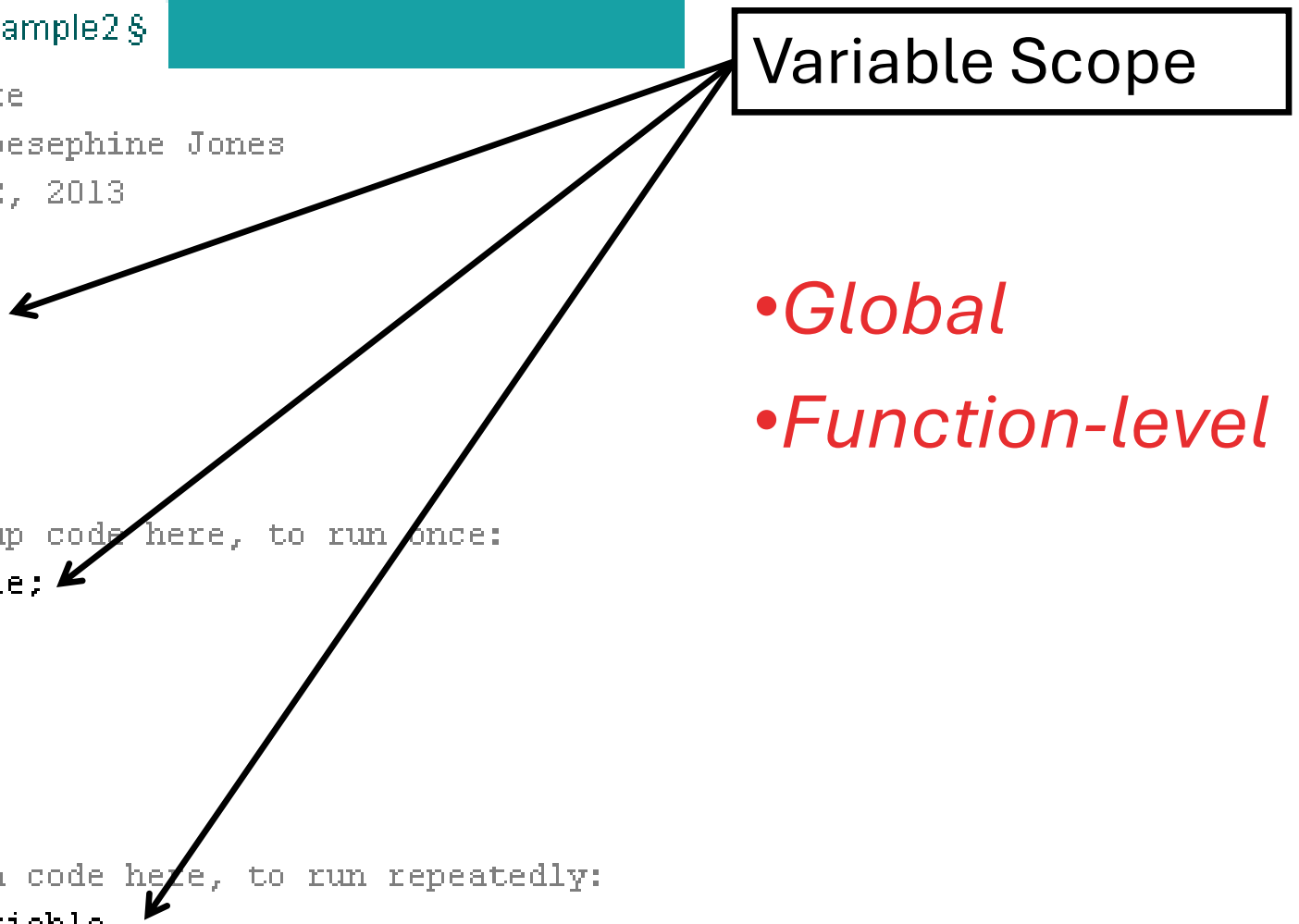
```
// Comments go here
// Written by:  Joesephine Jones
// Date:  April 12, 2013

int sensorValue;
int ledPin;

void setup()
{
  // put your setup code here, to run once:
  int setupVariable;

}

void loop()
{
  // put your main code here, to run repeatedly:
  int loopScopeVariable
}
```



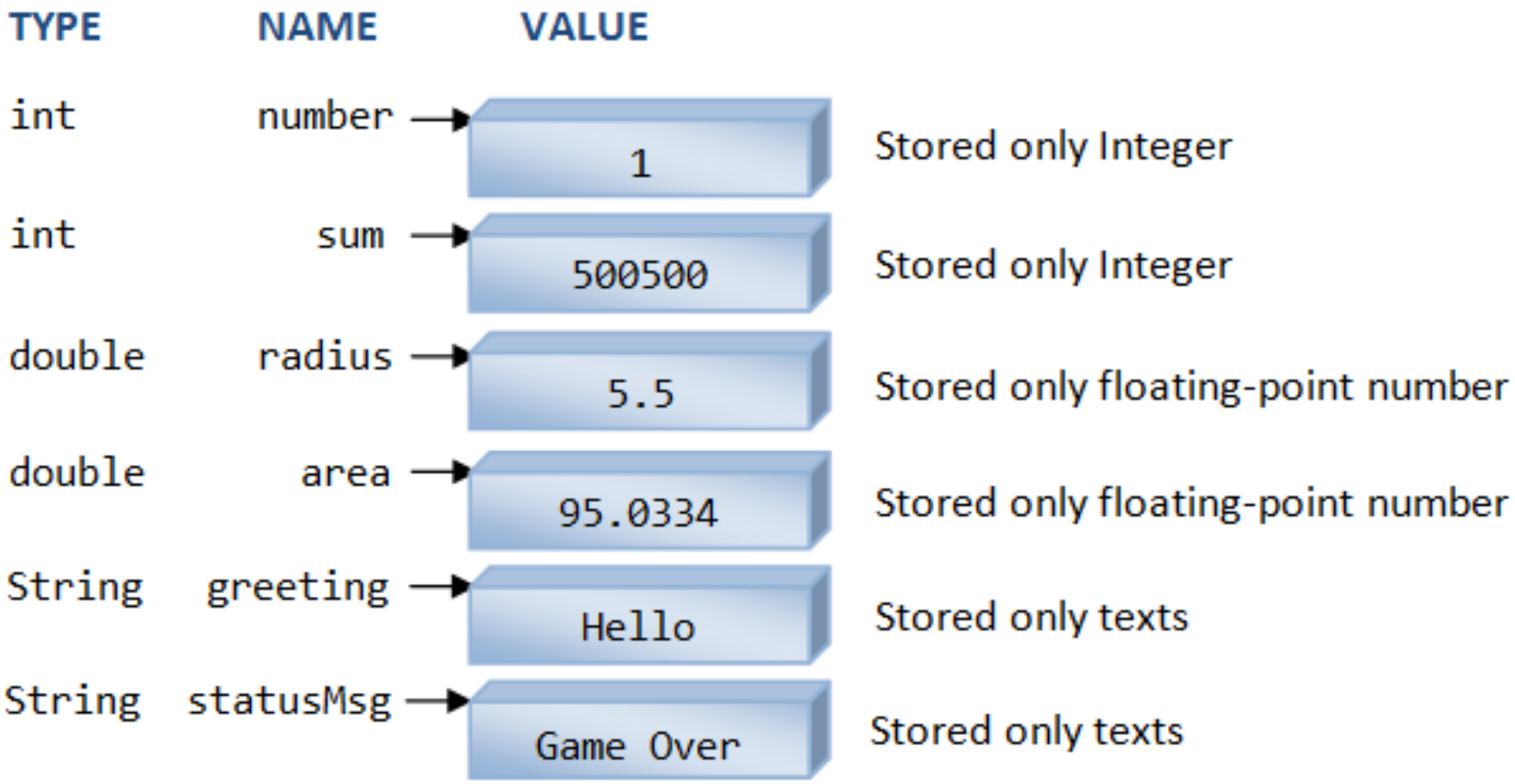
Variable Types

- **Variable Types:**



8 bits	16 bits	32 bits
byte char	int unsigned int	long unsigned long float

Variables



A *variable* has a ***name***, stores a ***value*** of the declared ***type***.

Format of variables

All variables have to be **declared** before they are used.

Declaring a variable means **defining its type**, and optionally, setting an initial value (initializing the variable).

For example,

```
int inputVariable = 0;
```

declares that variable **`inputVariable`** is of type **`int`**, and that its initial **`value is zero`**.

Format of variables

1. **`char`** – a data type that takes up 1 byte of memory that stores a character value. Character literals are written in single quotes, like this: 'A'.

2. **`byte`** – a byte stores an 8-bit unsigned number, from 0 to 255.

3. **`int`** – integers are your primary data type for number storage, and store a 2 byte value. This yields a range of −32,768 to 32,767.

4. **`unsigned int`**– unsigned integers are the same as integers in that they store a 2 byte value. Instead of storing negative numbers however they only store positive values, yielding a useful range of 0 to 65,535.

Format of variables

5. **`long`** – long variables are extended size variables for number storage, and store 32 bits (4 bytes), from −2,147,483,648 to 2,147,483,647.

6. **`unsigned long`** – unsigned long variables are extended size variables for number storage, and store 32 bits (4 bytes). Unlike standard longs unsigned longs won't store negative numbers, making their range from 0 to 4,294,967,295

Format of variables

7. **`Float`** – datatype for floating-point numbers, a number that has a decimal point. They are often used to approximate analog and continuous values because they have greater resolution than integers. Floating-point numbers can be as large as 3.4028235E+38 and as low as −3.4028235E+38. They are stored as 32 bits (4 bytes) of information.

8. **`double`** – double precision floating point number. Occupies 4 bytes. The double implementation on the Arduino is currently exactly the same as the float, with no gain in precision.

Format of variables

Example of variables:

```
char myChar = 'A';
char myChar = 65; // both are equivalent

byte b = B10010;  // "B" is the binary formatter
                        (B10010 = 18 decimal)

int ledPin = 13;

unsigned int ledPin = 13;
```

6. Arduino data types

Numeric types	Bytes	Range	Use
`int`	2	-32768 to 32767	Represents positive and negative integer values.
`unsigned int`	2	0 to 65535	Represents only positive values; otherwise, similar to `int`.
`long`	4	-2147483648 to 2147483647	Represents a very large range of positive and negative values.
`unsigned long`	4	4294967295	Represents a very large range of positive values.

Numeric types	Bytes	Range	Use
`float`	4	3.4028235E+38 to -3.4028235E+38	Represents numbers with fractions; use to approximate real-world measurements.
`double`	4	Same as `float`	In Arduino, `double` is just another name for `float`.
`boolean`	1	`false` (0) or `true` (1)	Represents true and false values.
`char`	1	-128 to 127	Represents a single character. Can also represent a signed value between -128 and 127.
`byte`	1	0 to 255	Similar to `char`, but for unsigned values.

Other types	
`string`	Represents arrays of `chars` (characters) typically used to contain text.
`void`	Used only in function declarations where no value is returned.

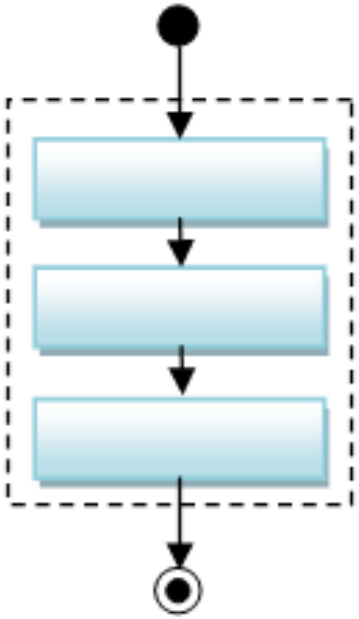
7. Arithmetic Operators

Math	Description
=	Assignment : makes something equal to something
%	Modulo : this gives the remainder when one number is divided by another
+	Addition
-	Subtraction
*	Multiplication
/	Division

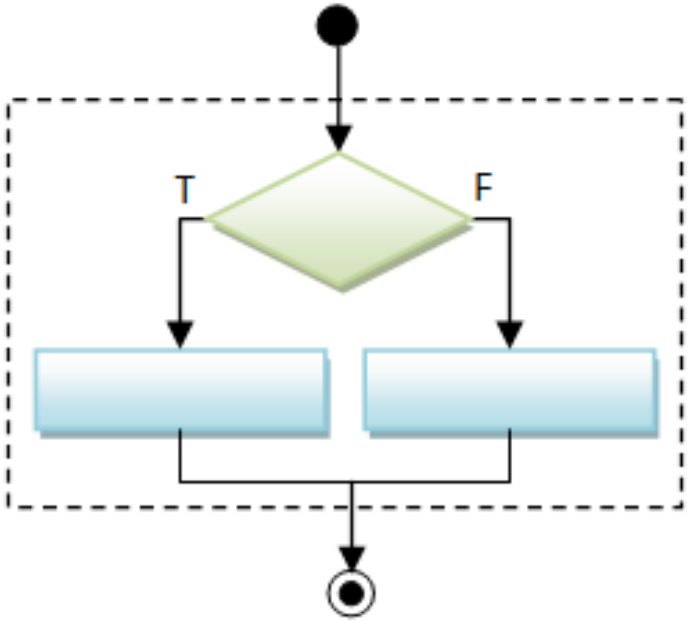
8. Comparison Operators

Comparison	Description
( ) == ( )	is equal?
( ) != ( )	is not equal?
( ) > ( )	greater than
( ) >= ( )	greater than or equal
( ) < ( )	less than
( ) <= ( )	less than or equal

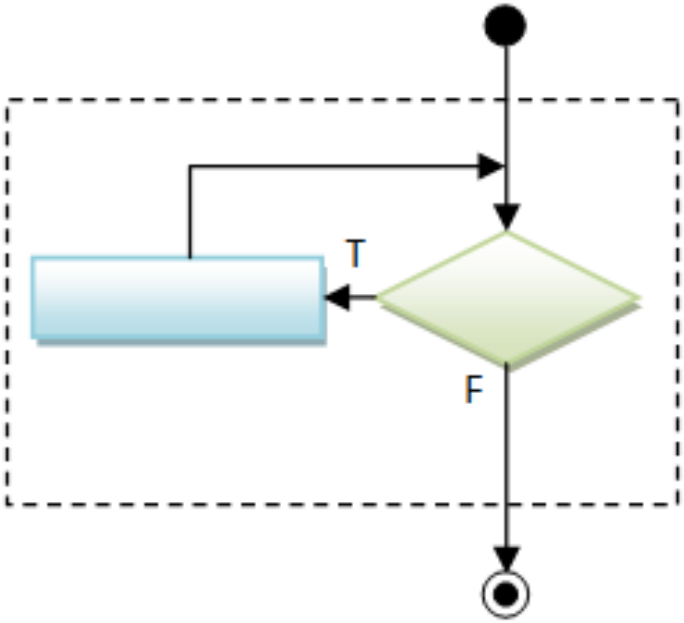
9. Flow Control



Sequential



Conditional (Decision)

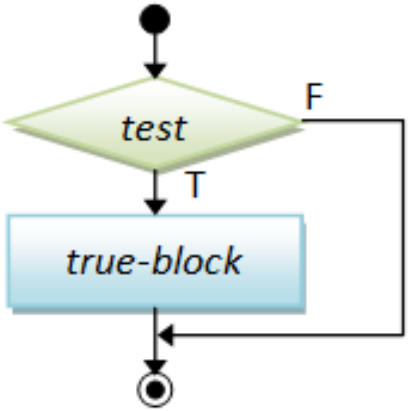


Loop (Iteration)

Condition Flow Control

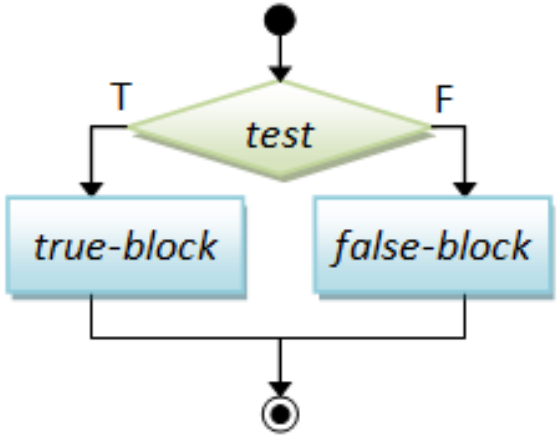
1. if- then

```
if(expression)
{
        True Block;
}
```



2. if- then-else

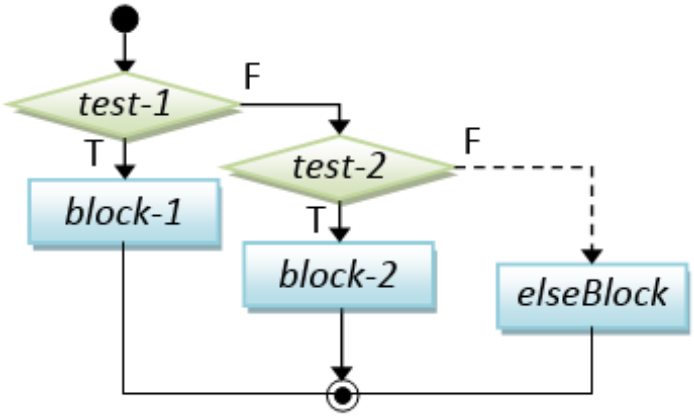
```
if(expression)
{
        True Block;
} else
{
        False Block;
}
```



10. Condition Flow Control

3. nested-if

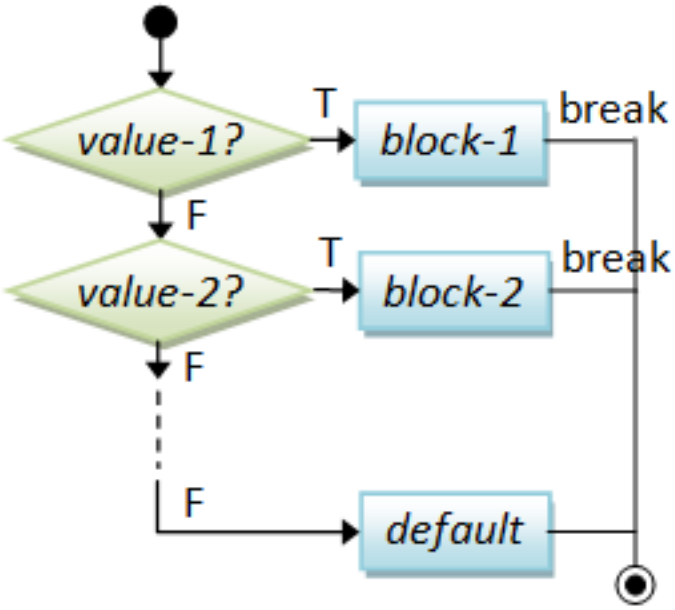
```
if(expression 1)
{
      True Block-1;
} else if(expression 2)
{
      True Block-2;
} else if(expression 3)
{
...
} else
{
      else Block
}
```



Condition Flow Control

4. switch-case

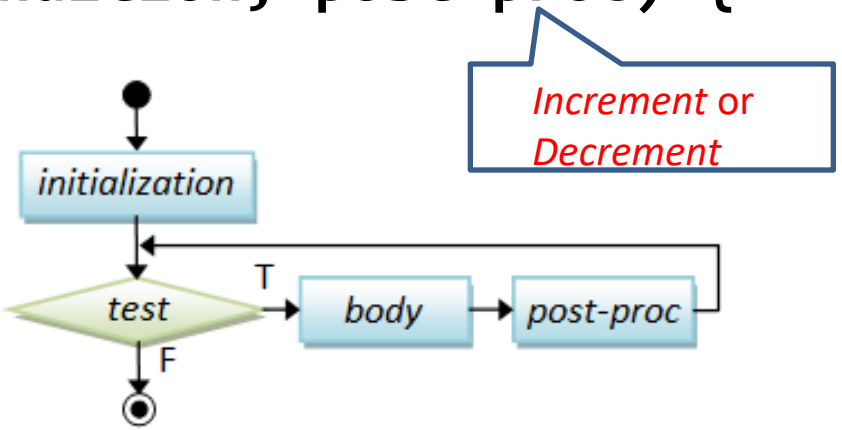
```
switch(selector) {
        case value-1:
                Block-1;
                break;
        case value-2:
                Block-2;
                break;
        case value-3:
                Block-3;
                break;
        ......
        default
                Default Block
                break;
}
```



11. Loop Flow Control

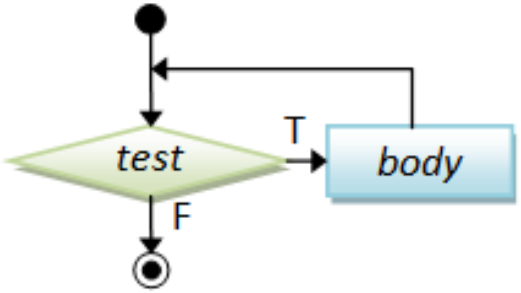
1. for

```
for (initial; test-condition; post-proc) {
        body;
}
```



2. while

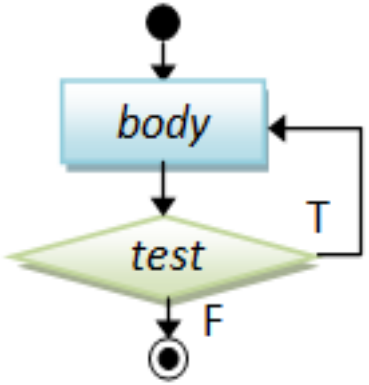
```
while(expression)
{
        body;
}
```



Loop Flow Control

3. do.. while

```
do
{
        Body;
}
while (expression);
```



12. Arduino Function

Language is standard C (but made easy)

core function

Digital I/O

- pinMode()
- digitalWrite()
- digitalRead()

Analog I/O

- analogReference()
- analogRead()
- analogWrite() - *PWM*

Communication

- Serial

Advanced I/O

- tone()
- noTone()
- shiftOut()
- shiftIn()
- pulseIn()

Time

- millis()
- micros()
- delay()
- delayMicroseconds()

Math

- min()
- max()
- abs()
- constrain()
- map()
- pow()
- sqrt()

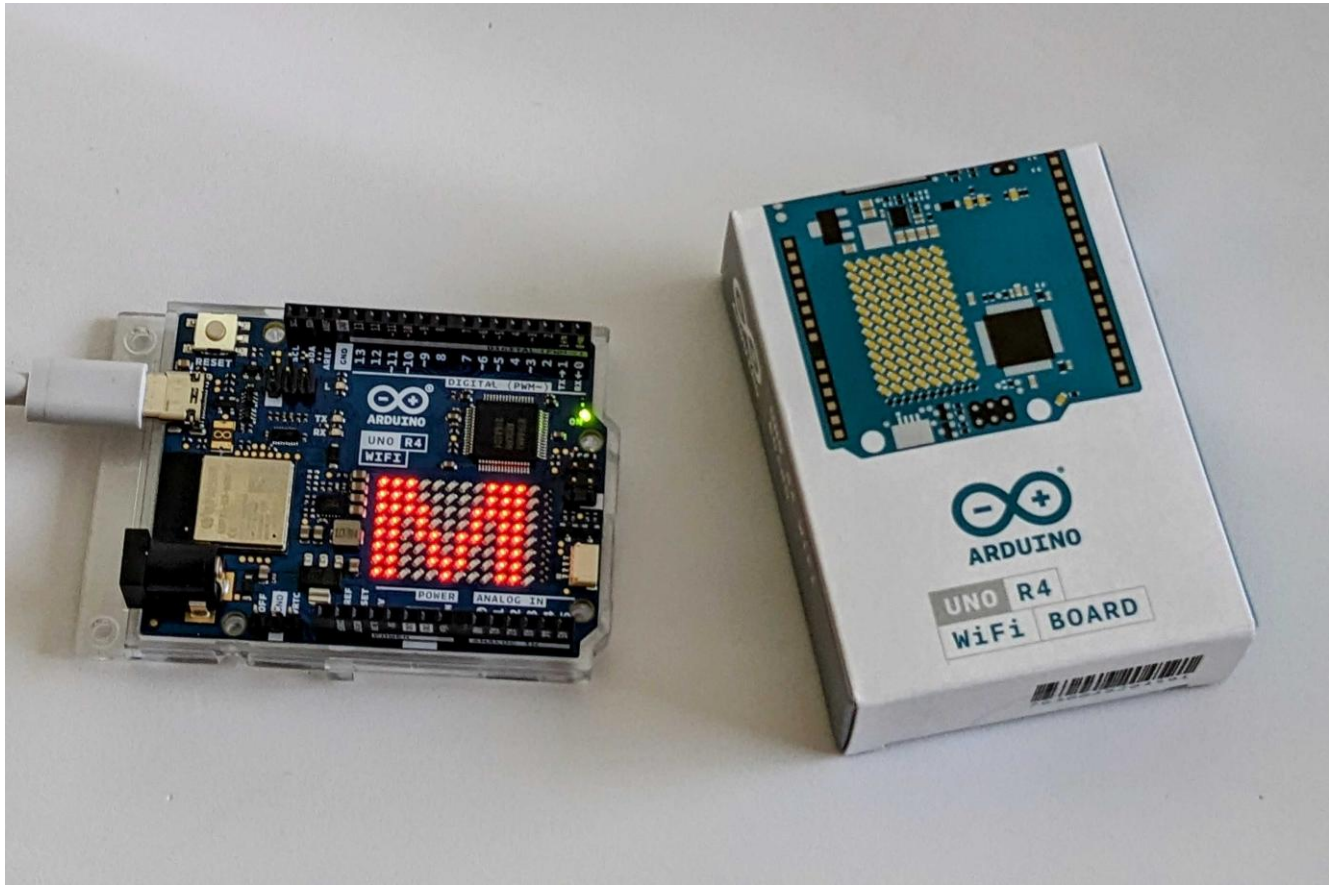
Trigonometry

- sin()
- cos()
- tan()

Random Numbers

- randomSeed()
- random()

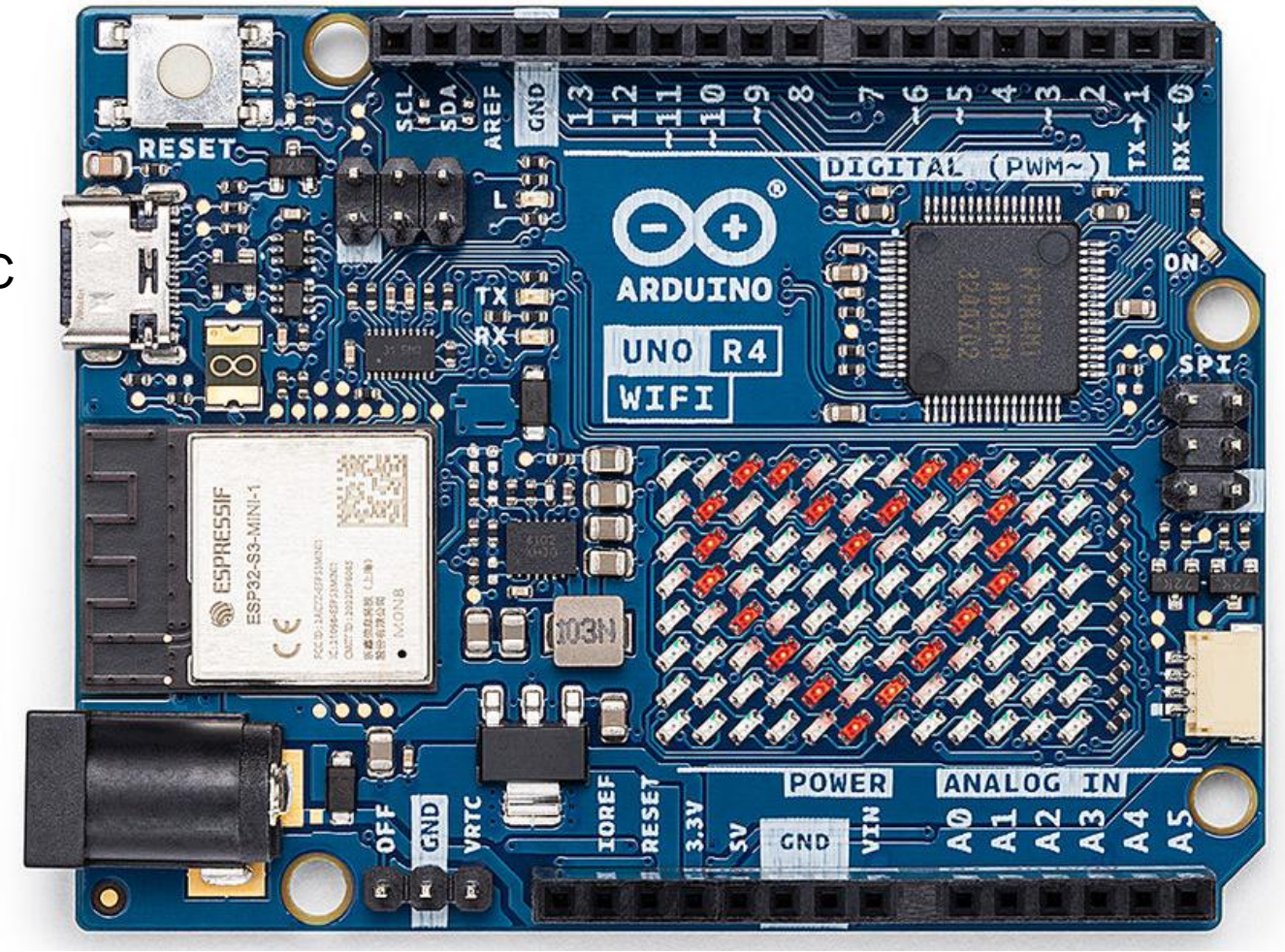
3. Laboratory Device



3.1 Arduino Uno Board

Digital Input / Output D0-D13

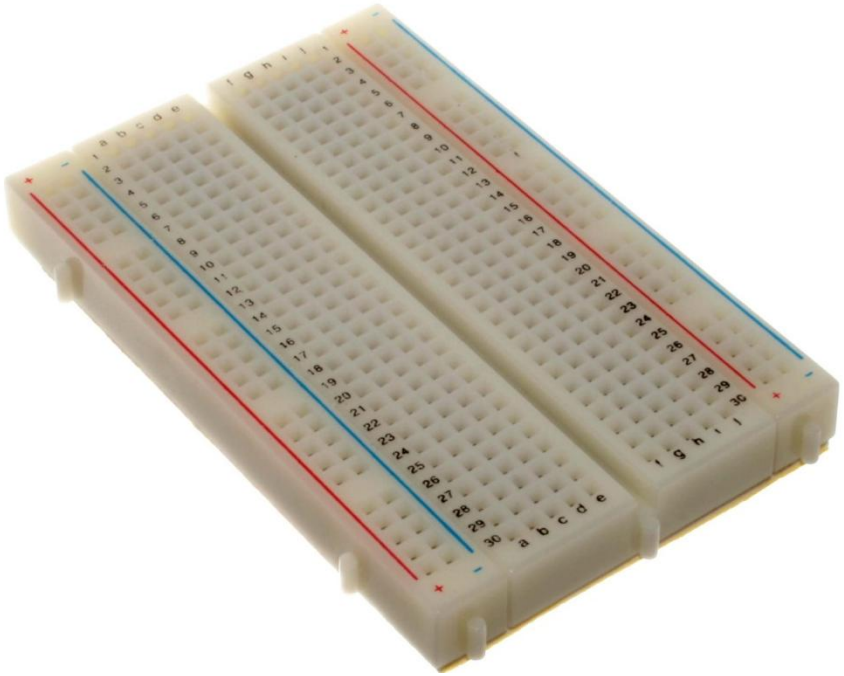
USB Type C

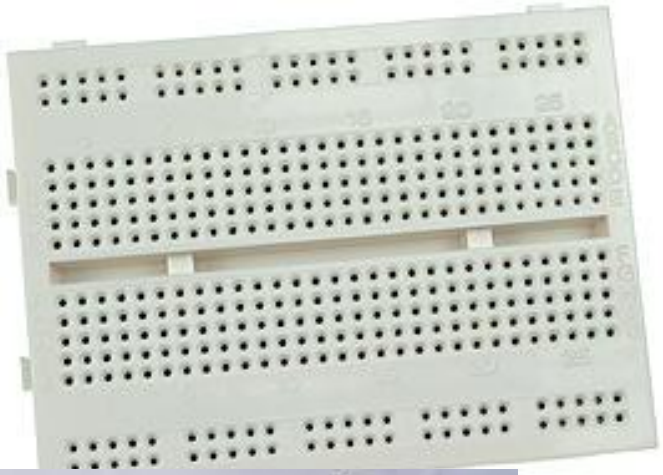


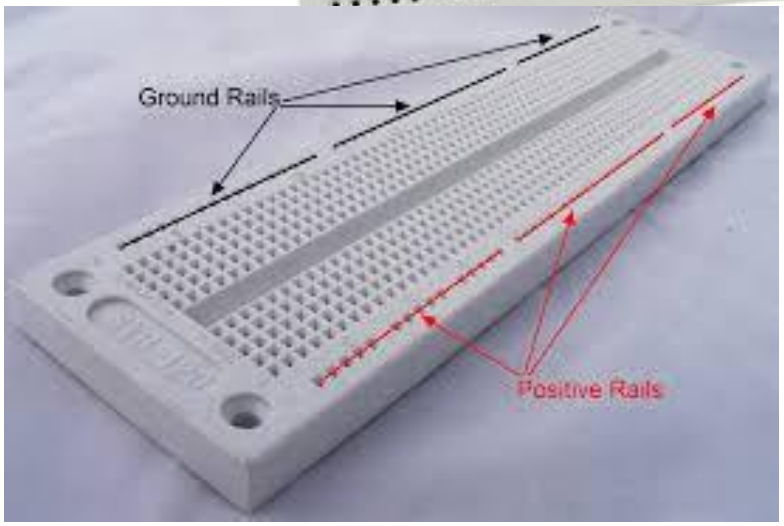
12 x 8
LED Matrix

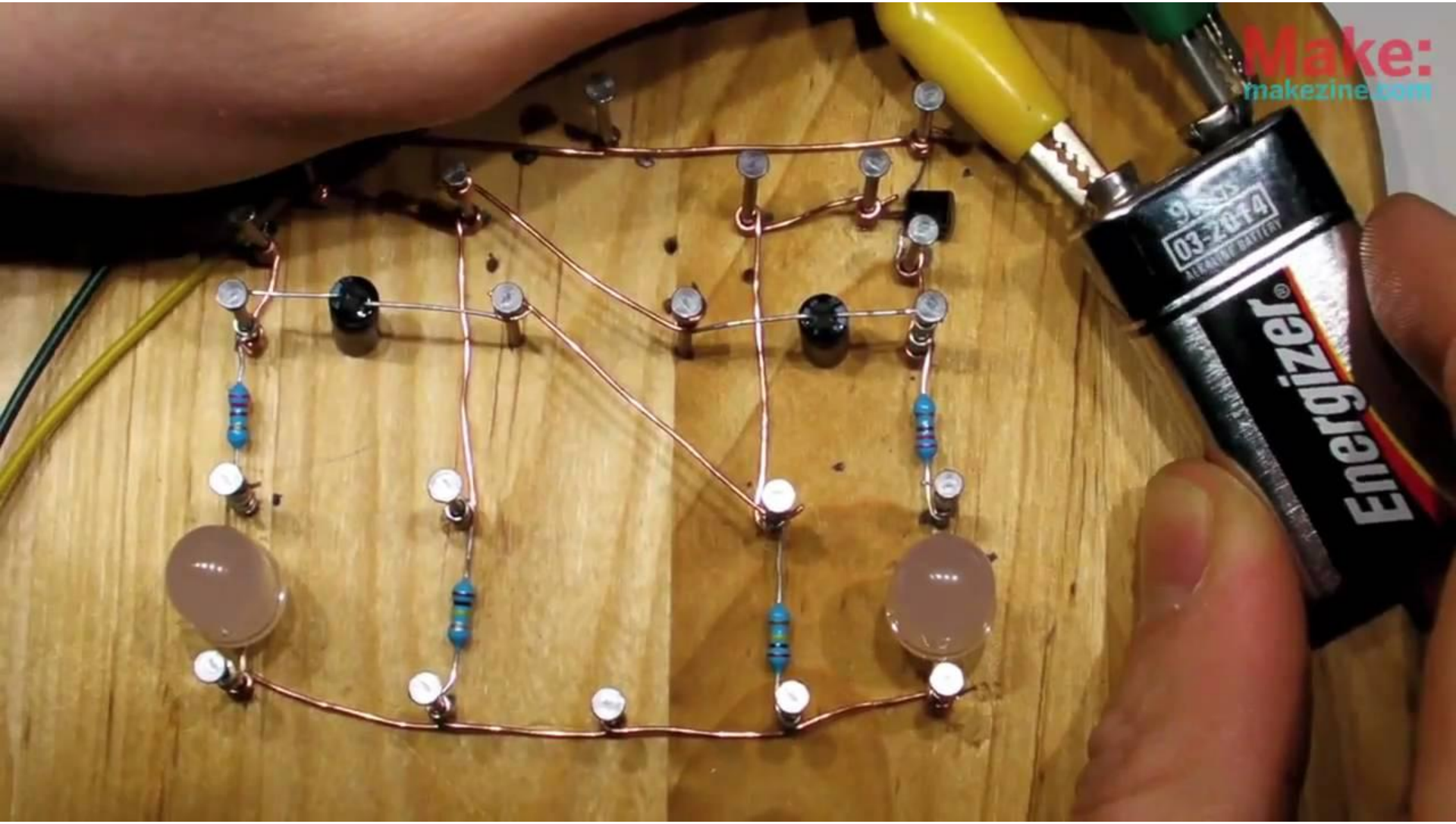
Analog Input
(ADC)

3.2 Breadboard

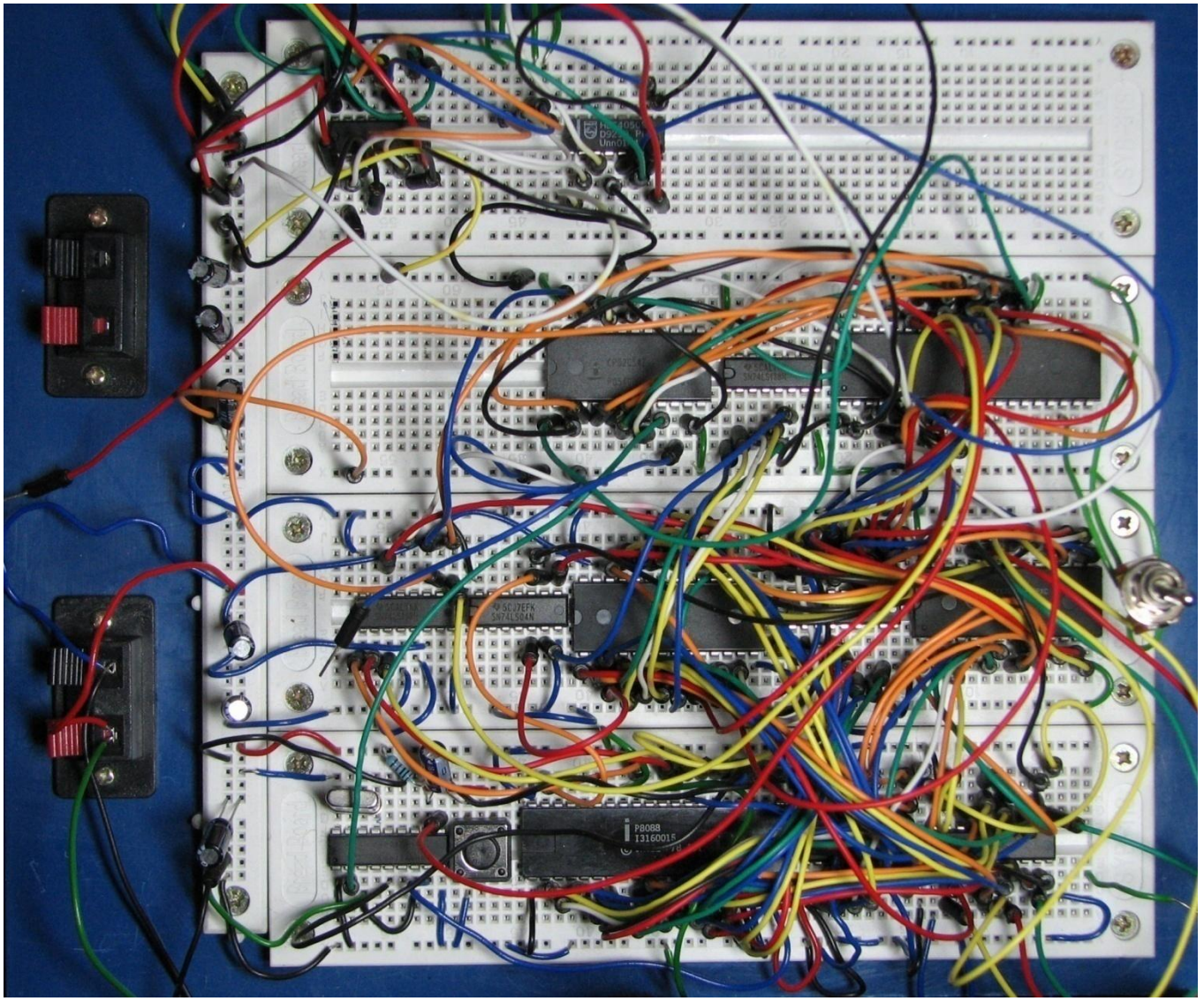




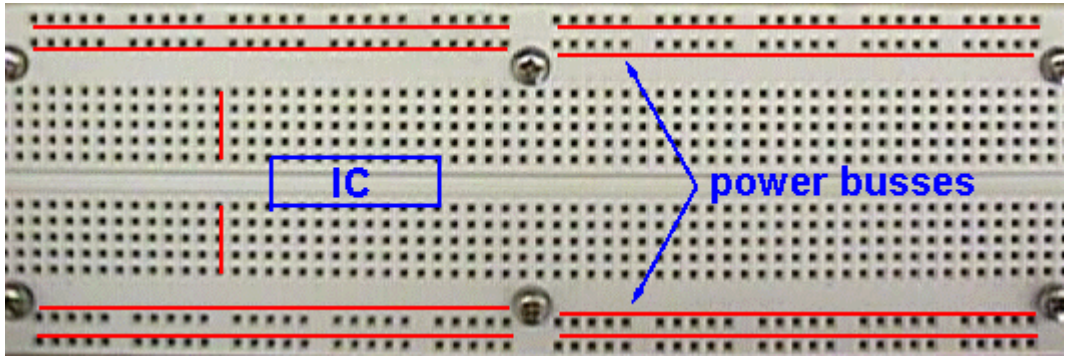


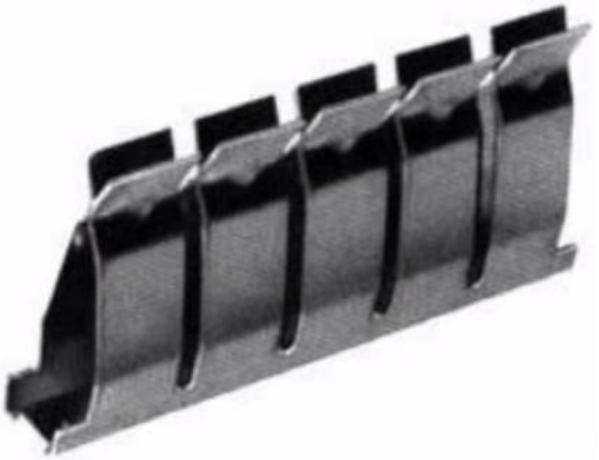

Make:
makezine.com
03-2014
Energizer

What circuit are we going to breadboard today?



How does a breadboard work?

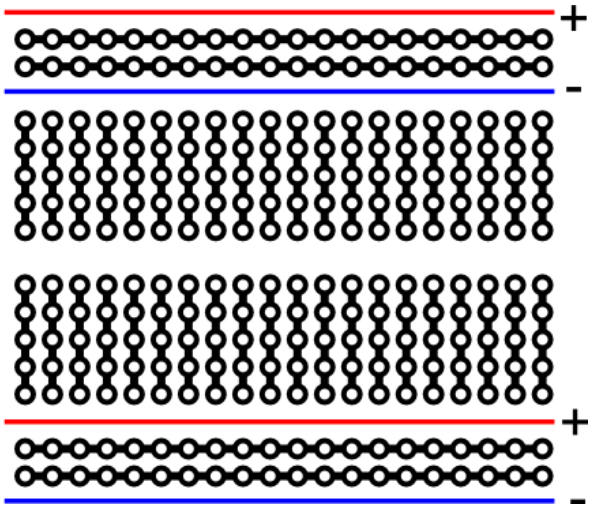


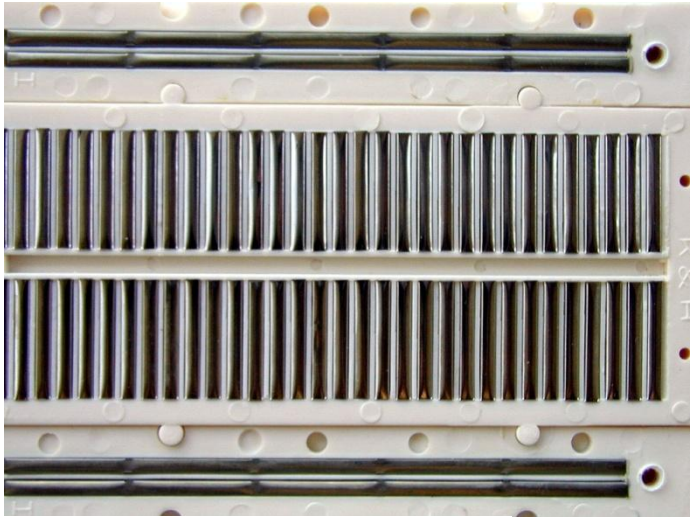


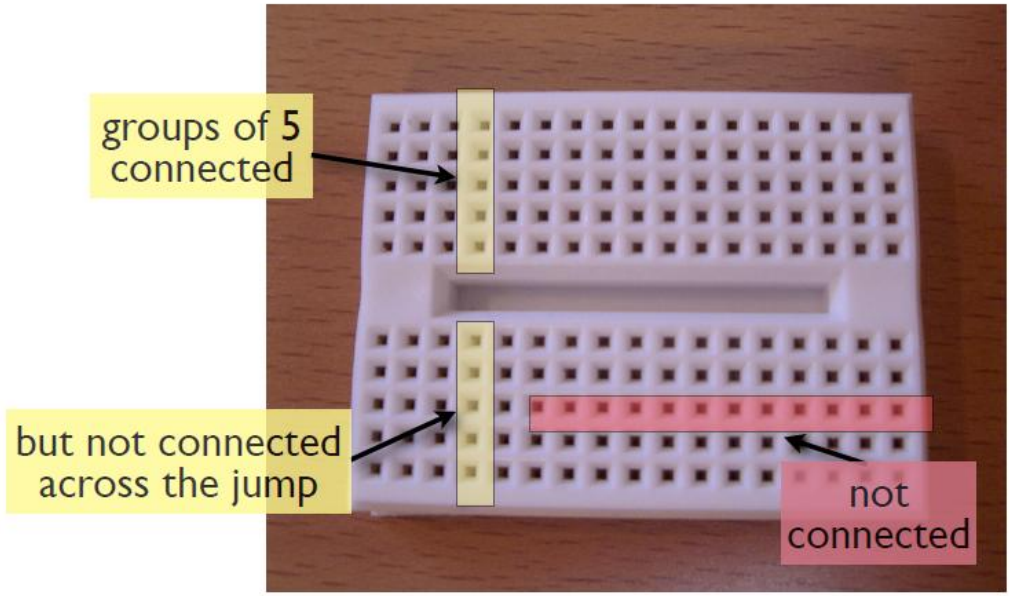




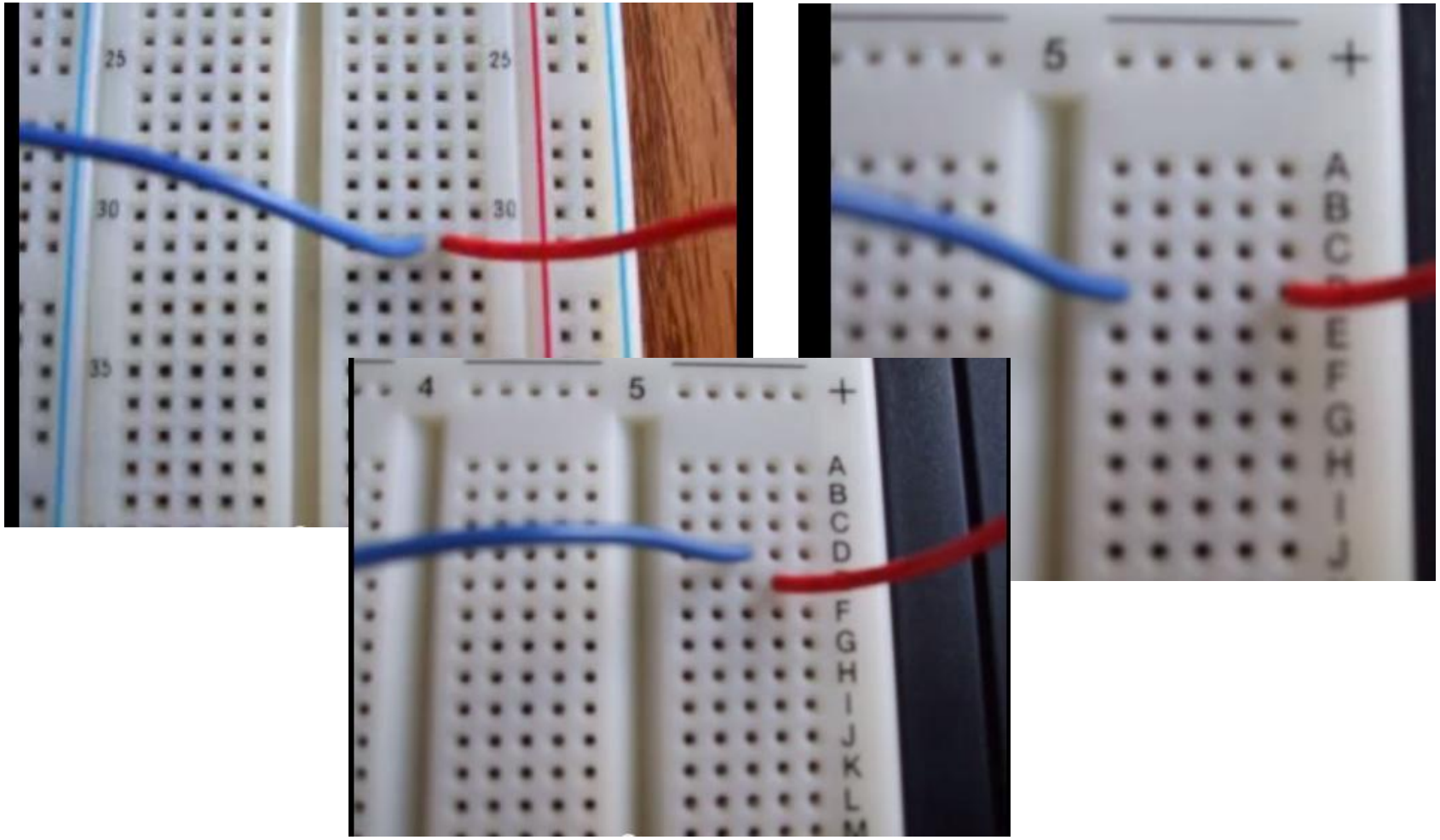
How does a breadboard work?



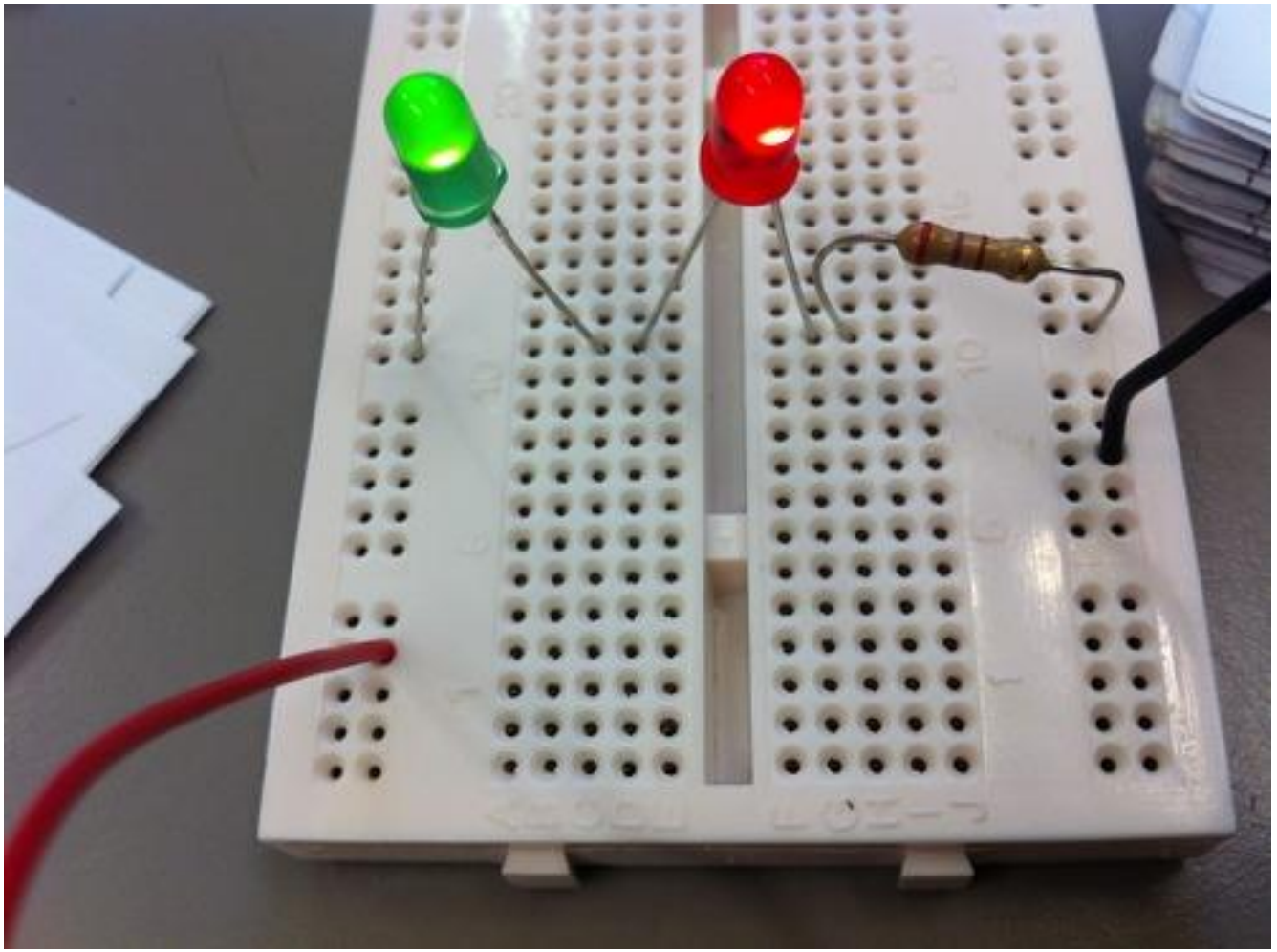




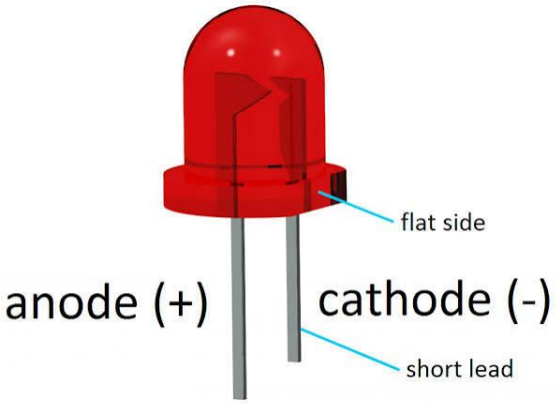
Are these wires connected?



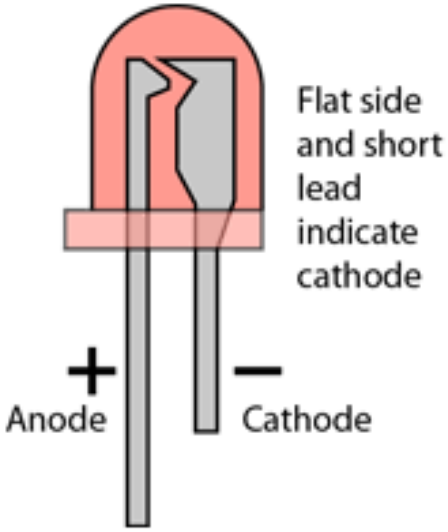
How to use a breadboard



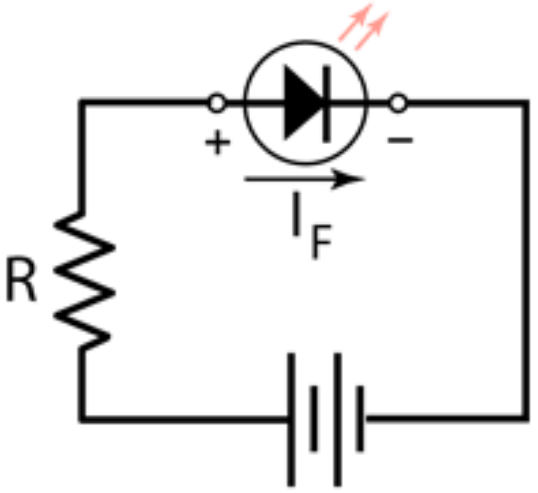
3.3 LED



- LED = Light Emitting Diode
- Need a “current limit” resistor or burns out.

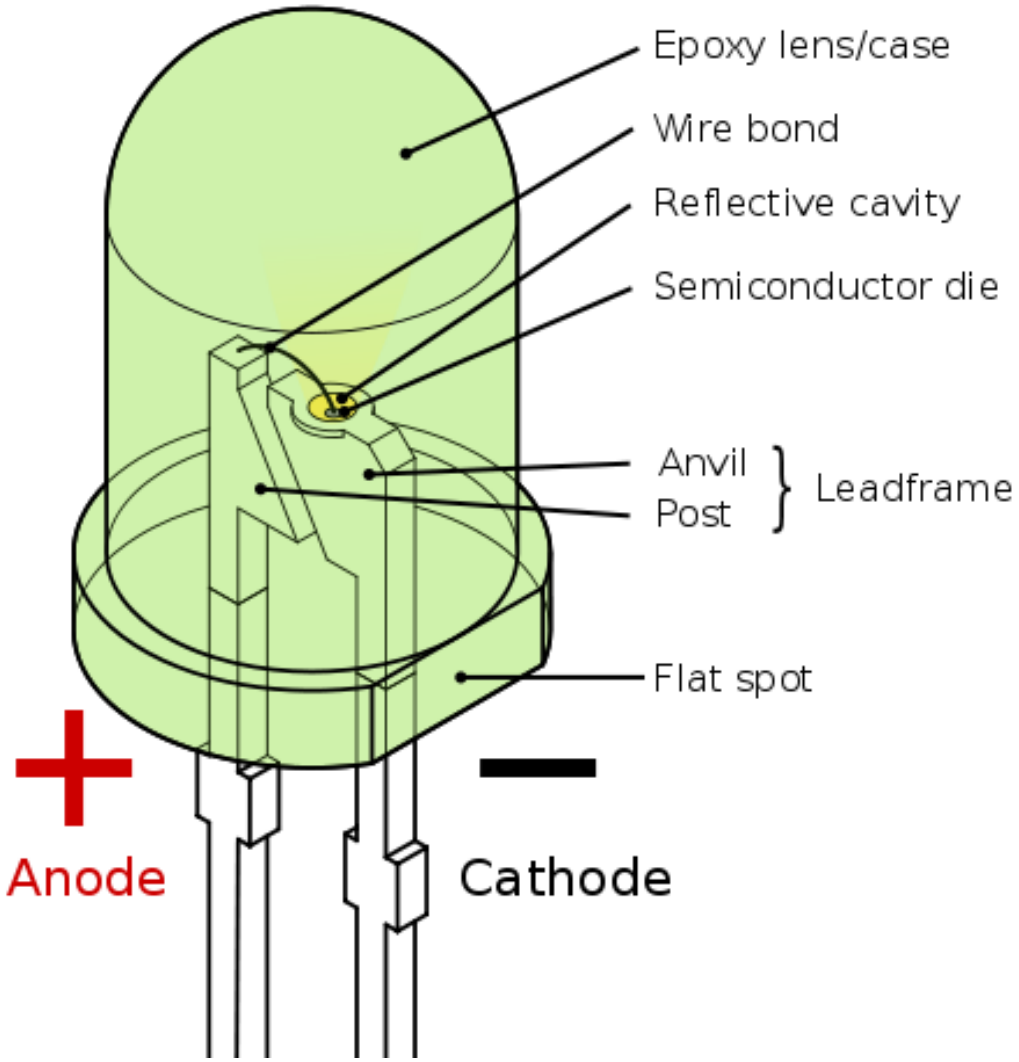




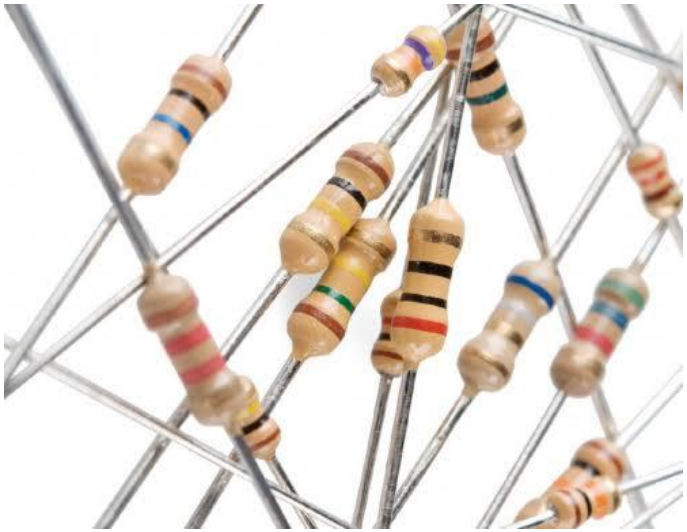


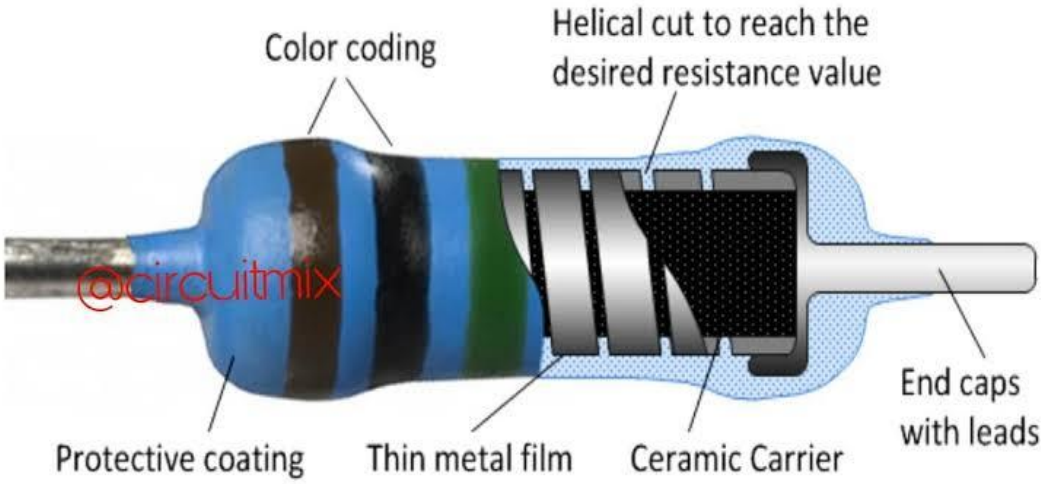
Schematic Symbol

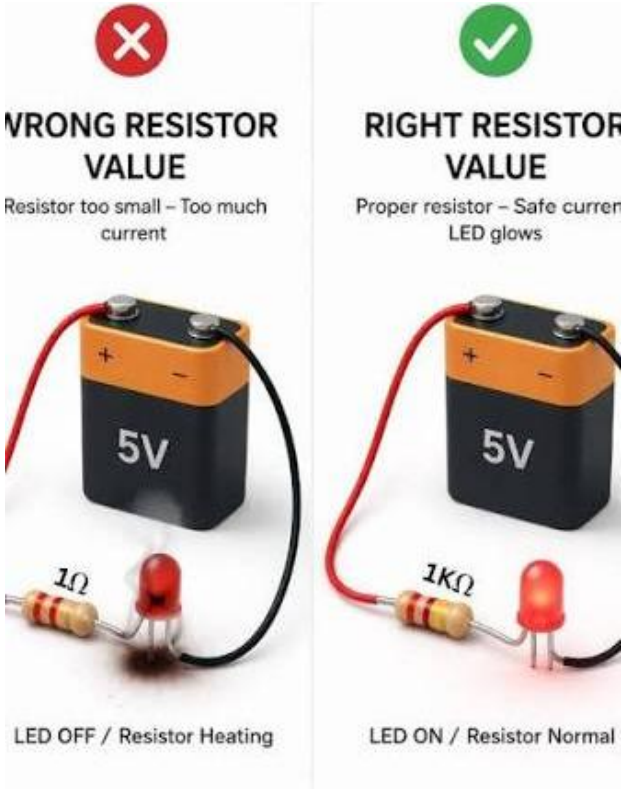
What are LEDs?

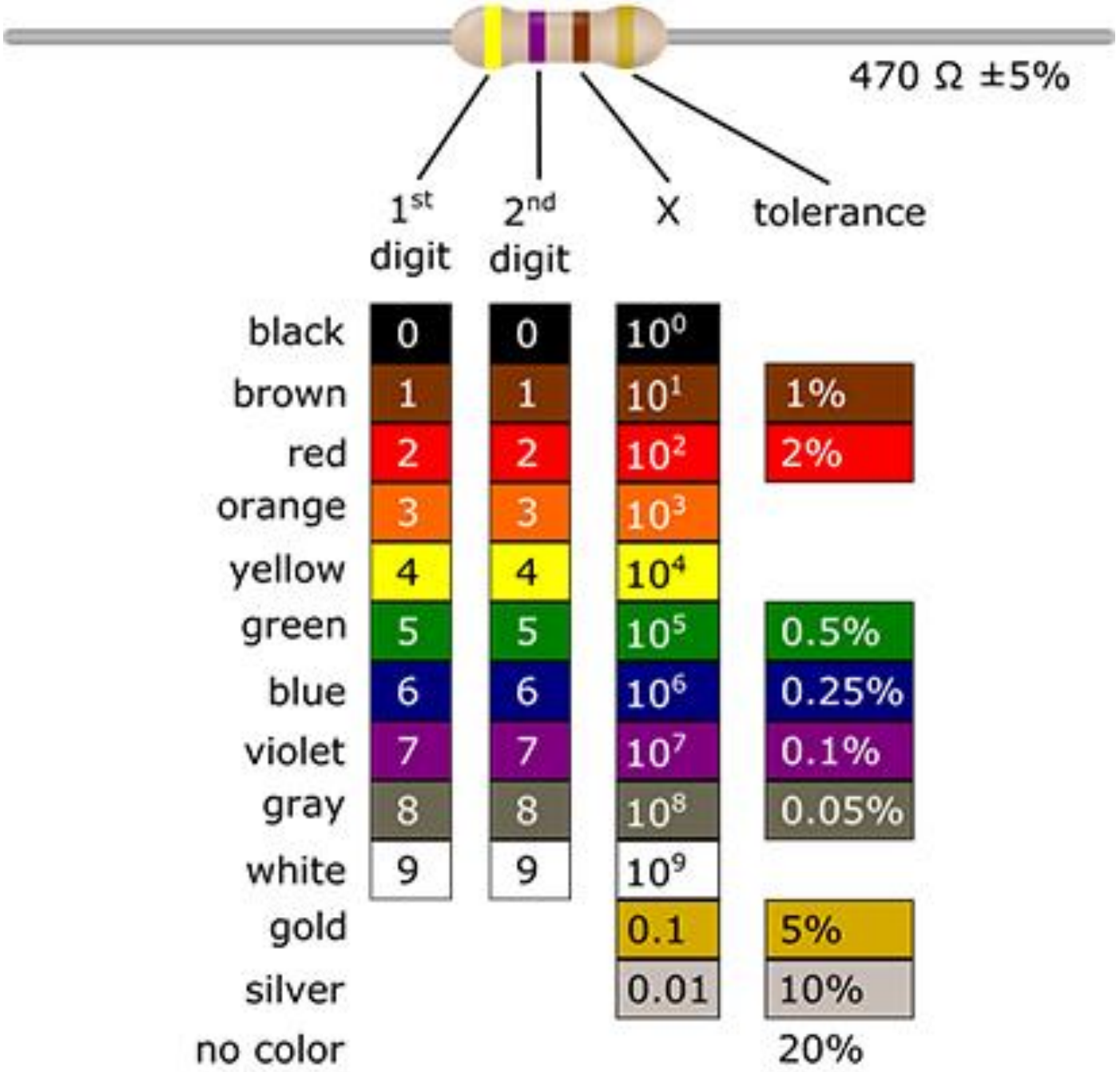


3.4 Resistor

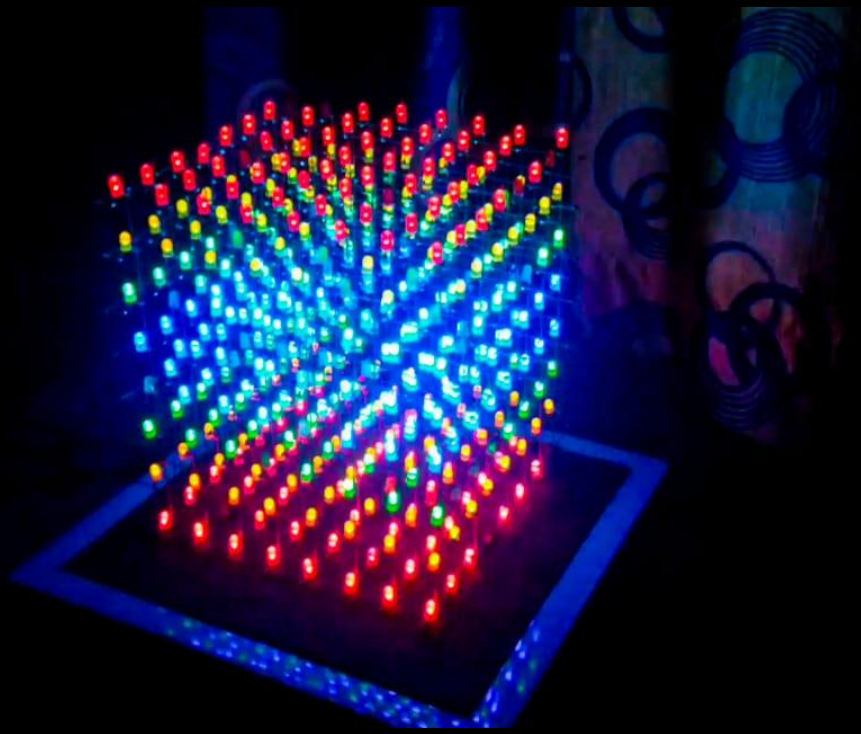



Color coding
Helical cut to reach the desired resistance value
@circuitmix
Protective coating
Thin metal film
Ceramic Carrier
End caps with leads


VRONG RESISTOR VALUE
Resistor too small – Too much current
5V
1Ω
LED OFF / Resistor Heating
RIGHT RESISTOR VALUE
Proper resistor – Safe curren
LED glows
5V
1KΩ
LED ON / Resistor Normal


470 Ω ±5%
1st digit
2nd digit
X
tolerance
black 0 0 10^0
brown 1 1 10^1 1%
red 2 2 10^2 2%
orange 3 3 10^3
yellow 4 4 10^4
green 5 5 10^5 0.5%
blue 6 6 10^6 0.25%
violet 7 7 10^7 0.1%
gray 8 8 10^8 0.05%
white 9 9 10^9
gold 0.1 5%
silver 0.01 10%
no color 20%

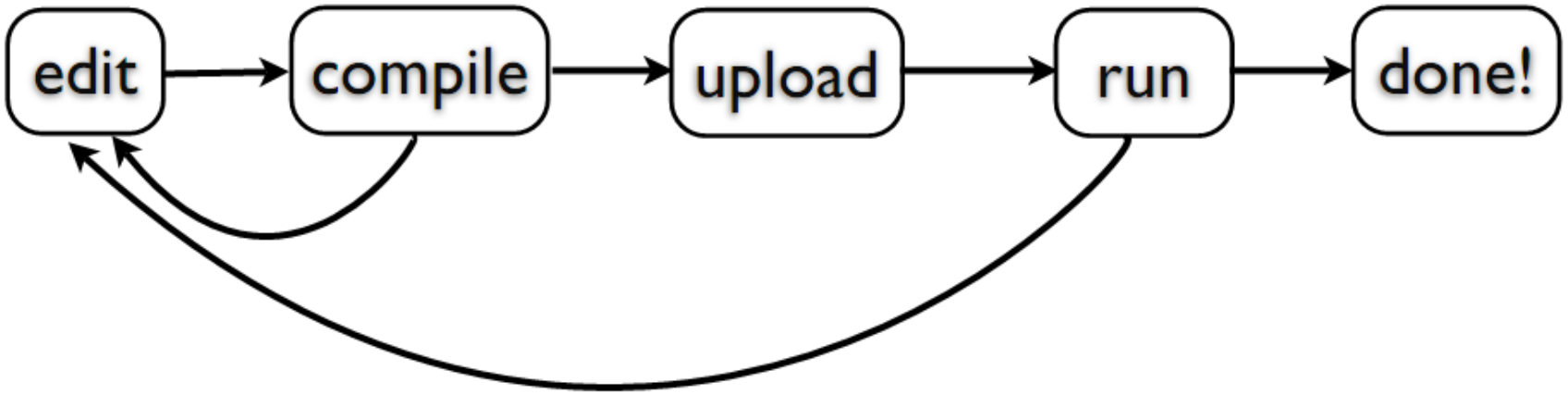
4. การส่งข้อมูลออกพอร์ต Digital output



arduino cube

1. ขั้นตอนการพัฒนาโปรแกรมด้วย Arduino (Development Circle)

- Make as many changes as you want
- Not like most web programming: edit → run
- Edit → compile → upload → run



2. Some useful function

- **`pinMode()`** – set a pin as input or output
- **`digitalWrite()`** – set a digital pin high/low
- **`digitalRead()`** – read a digital pin's state
- **`analogWrite()`** – write an "analog" value
- **`analogRead()`** – read an analog pin
- **`delay()`** – wait an amount of time
- **`millis()`** – get the current time

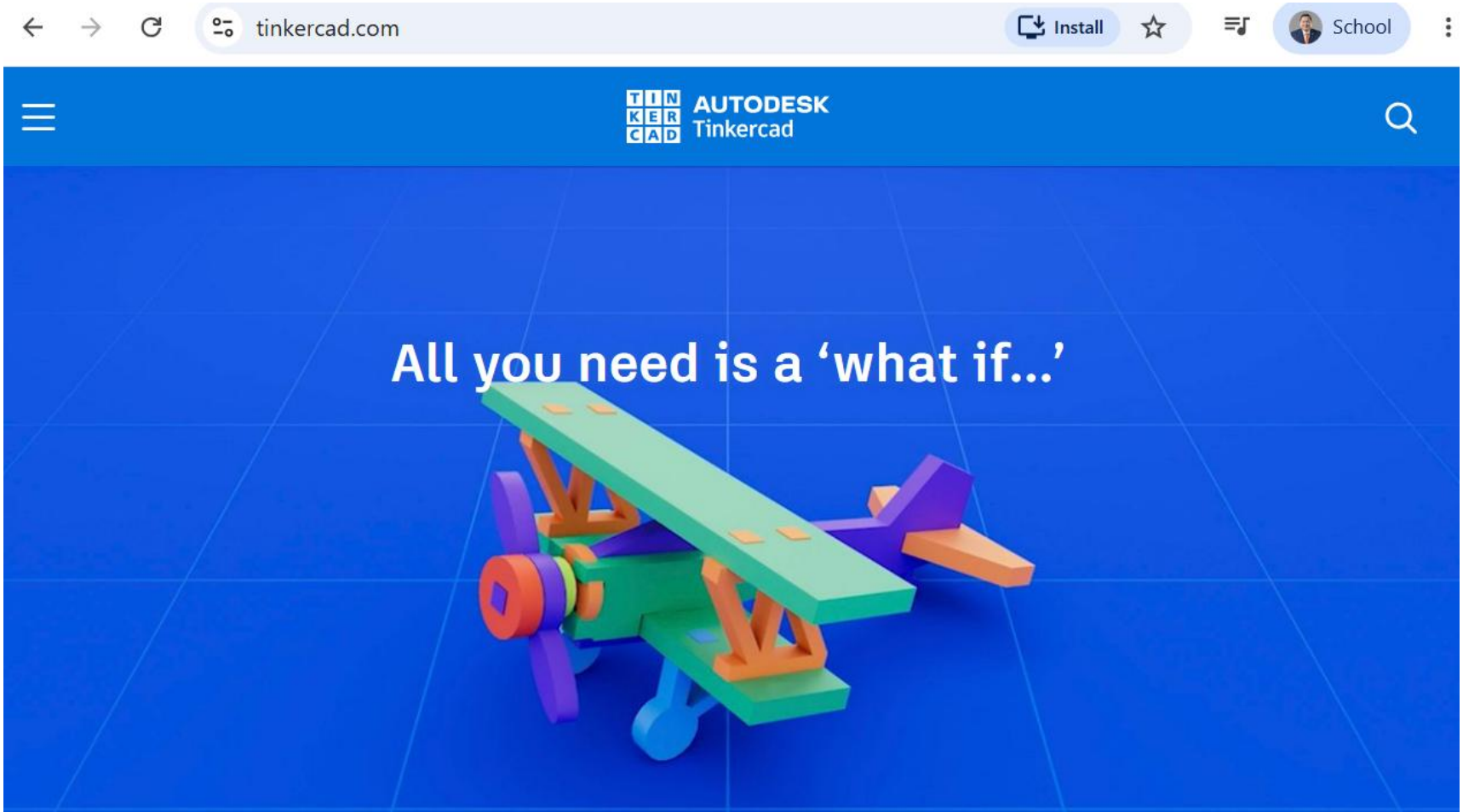
3. การเขียนโปรแกรมบน Arduino

การเขียนโปรแกรมบน Arduino จะคล้ายกับการเขียนโปรแกรม C/C++ ไฟล์โปรแกรม ที่ทำการเขียนจะเรียกว่า sketch files มีส่วนประกอบสามส่วน คือ

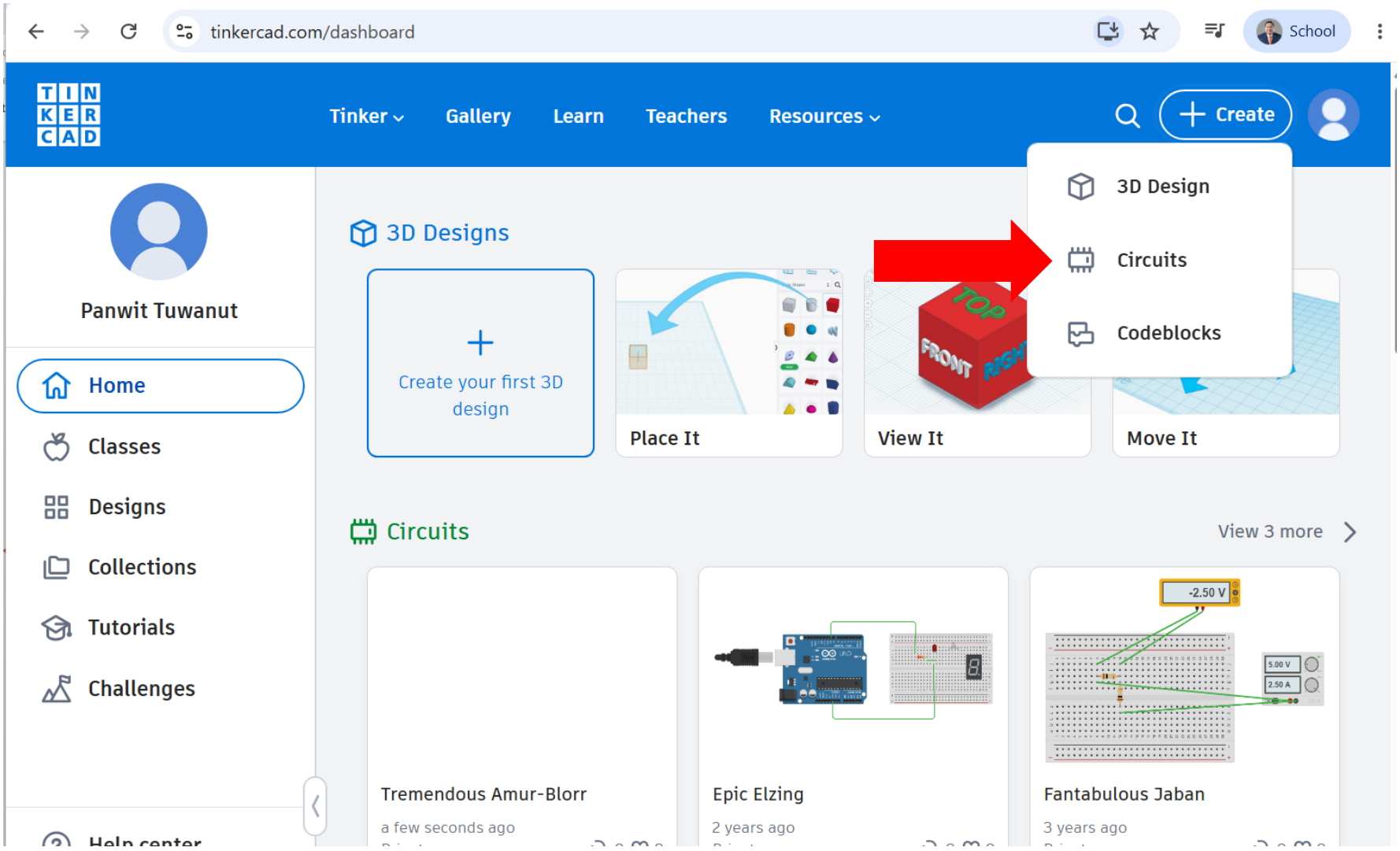
- Declare variables at top
- Initialize
  - `setup()` – run once at beginning, set pins
- Running
  - `loop()` – run repeatedly, after `setup()`

Laboratory
การทดลองที่ 1 Getting Start with Arduino

https://www.tinkercad.com/



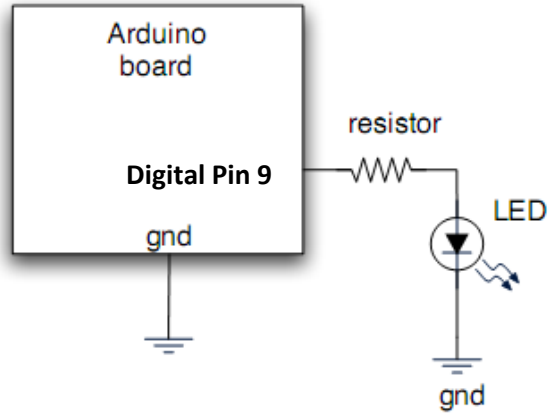
ทำการ sign in และ Log in



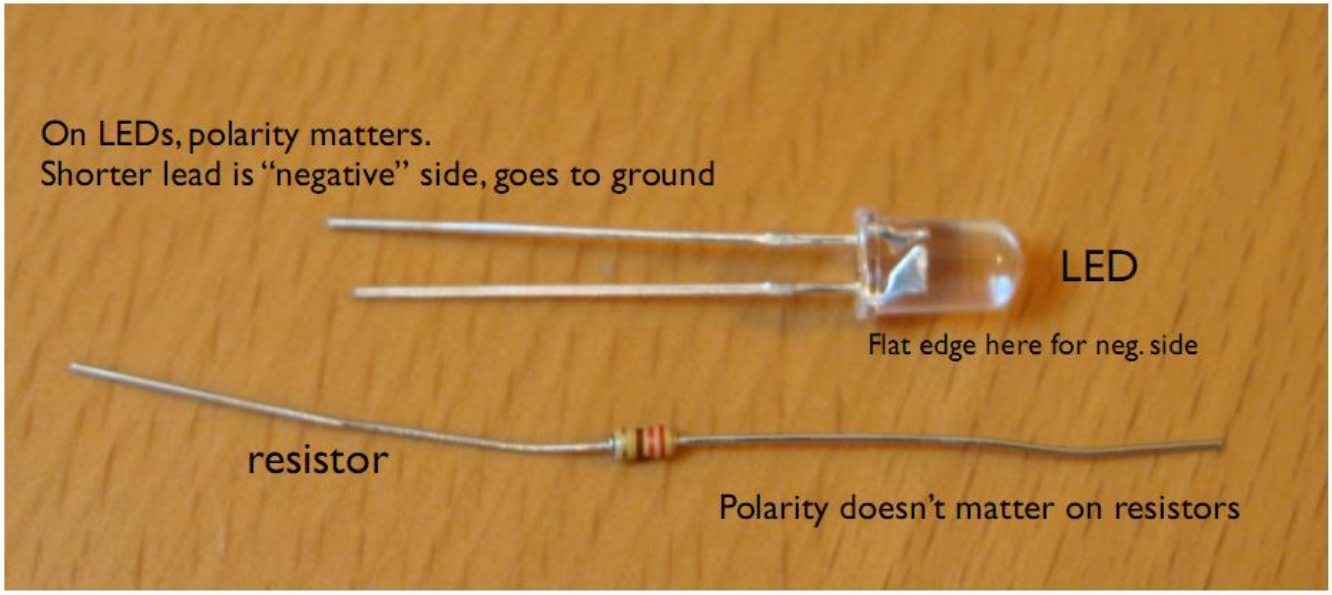
4. การส่งค่าออกพอร์ต แบบดิจิตอล

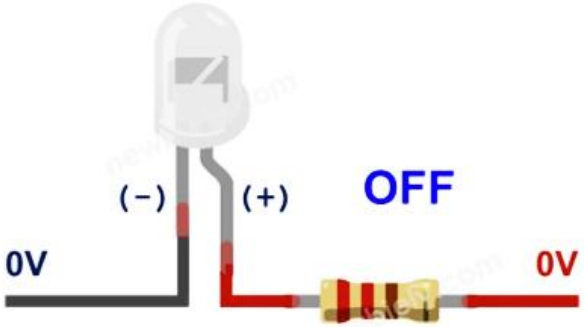
การทดลองที่ 1. LED Blink : Hello world of Microcontroller

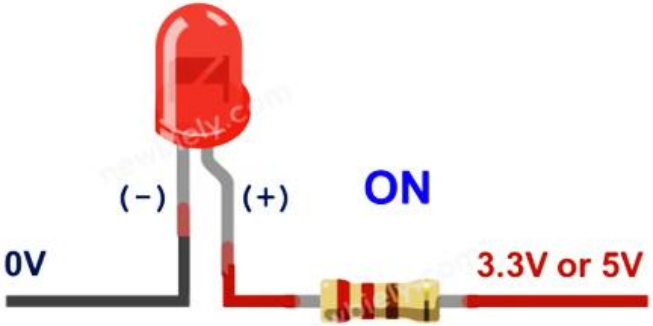
schematic

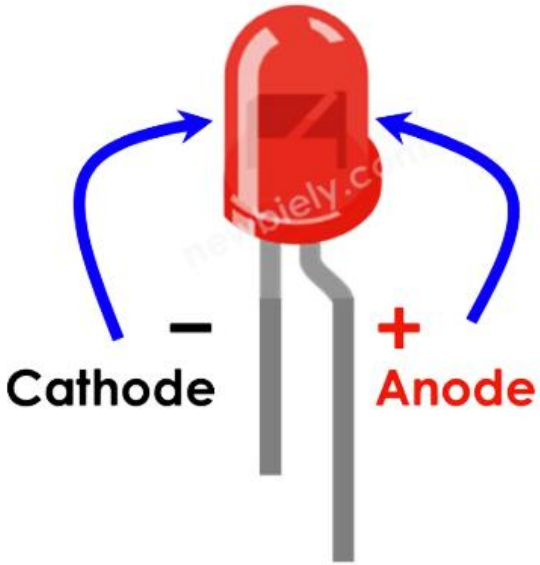


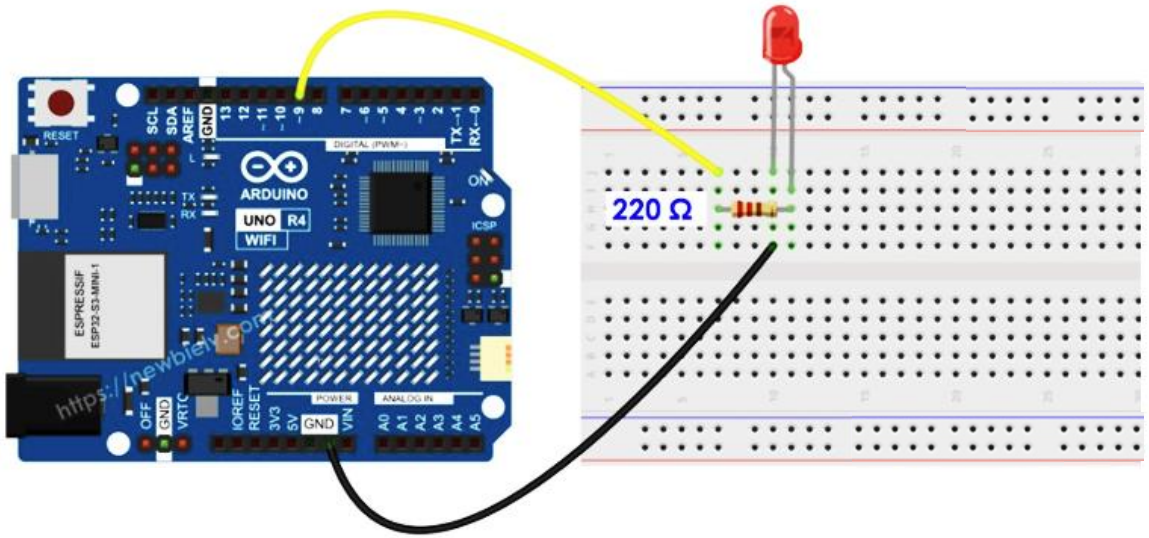
To turn on LED use
`digitalWrite(9,HIGH)`




(−)
(+)
OFF
0V
0V


(−)
(+)
ON
0V
3.3V or 5V


Cathode
Anode


ARDUINO
UNO R4
WIFI
220 Ω

โปรแกรม LED Blink

```
void setup() {
  // initialize digital pin LED_BUILTIN as an output.

  pinMode(9, OUTPUT);
}

// the loop function runs over and over again forever

void loop() {
  digitalWrite(9, HIGH);  // turn the LED on (HIGH is the voltage level)
  delay(1000);            // wait for a second
  digitalWrite(9, LOW);   // turn the LED off by making the voltage LOW
  delay(1000);            // wait for a second
}
```

แบบฝึกหัดที่ 1.1 โปรแกรมควบคุม Color LED

จงเขียนโปรแกรมแสดงสี 7 สี โดยเว้นช่วงสีละ 1 วินาที

กำหนดสีเอง ตามใจชอบ

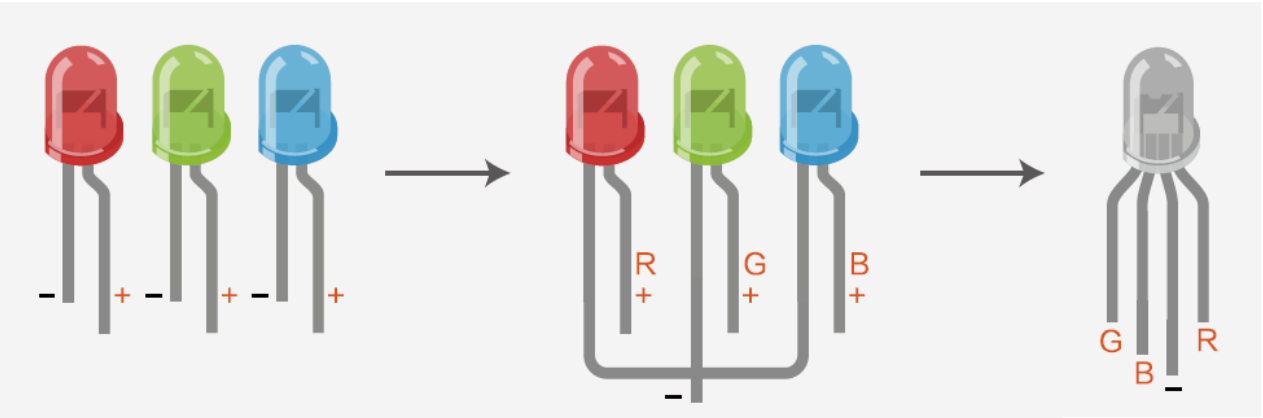


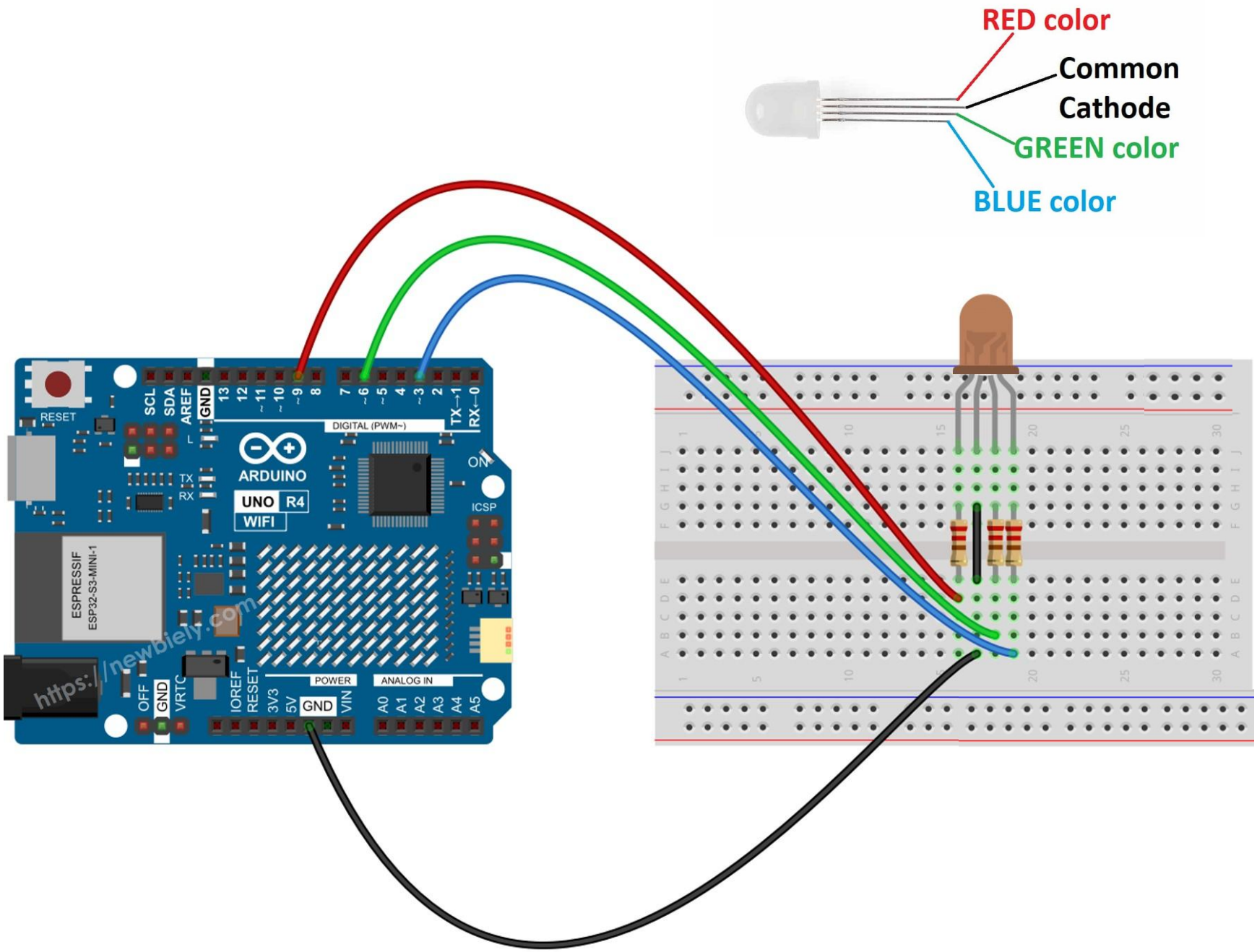
Fig. 5-4 Common Cathode RGB Diagram



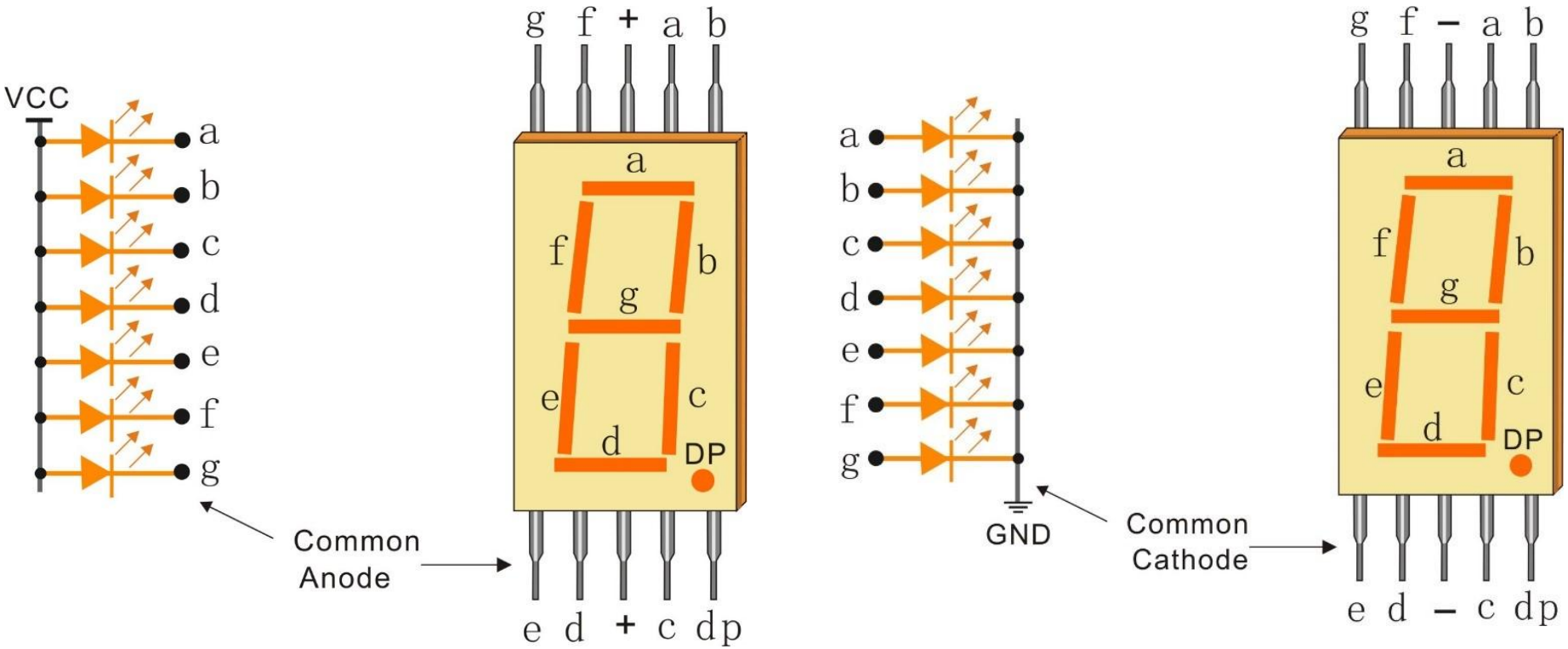
Fig. 5-5 Common Anode RGB Diagram



Common Cathode **Common Anode**


RED color
Common
Cathode
GREEN color
BLUE color
RESET
SCL
SDA
AREF
GND
DIGITAL (PWM~)
TX→1
RX←0
ARDUINO
UNO R4
WIFI
ON
ICSP
TX
RX
ESPRESSIF
ESP32-S3-MINI-1
https://newbiely.com
OFF
GND
VRTC
IOREF
RESET
3V3
5V
GND
VIN
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5

ประเภทของ 7-segment



แบบฝึกหัดที่ 1.2 โปรแกรมควบคุม 7-segment

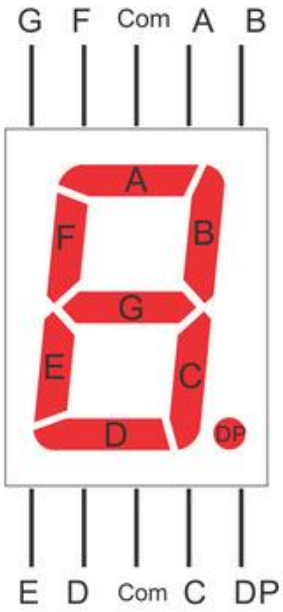
- ดัดแปลงวงจรไฟวิ่งในการทดลองที่ 1.1 โดยแทน LED ด้วย 7-segment

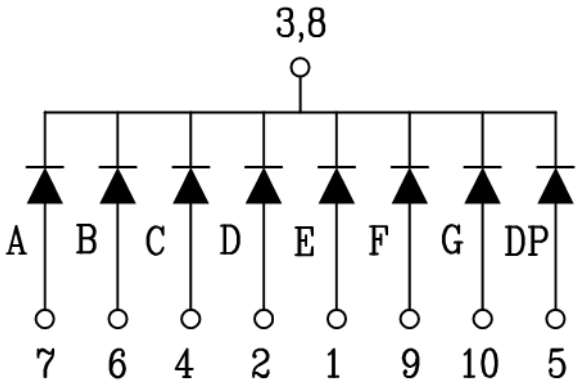
ATMega328P and Arduino Uno Pin Mapping

Arduino function				Arduino function
reset	(PCINT14/RESET) PC6	1	28 PC5 (ADC5/SCL/PCINT13)	analog input 5
digital pin 0 (RX)	(PCINT16/RXD) PD0	2	27 PC4 (ADC4/SDA/PCINT12)	analog input 4
digital pin 1 (TX)	(PCINT17/TXD) PD1	3	26 PC3 (ADC3/PCINT11)	analog input 3
digital pin 2	(PCINT18/INT0) PD2	4	25 PC2 (ADC2/PCINT10)	analog input 2
digital pin 3 (PWM)	(PCINT19/OC2B/INT1) PD3	5	24 PC1 (ADC1/PCINT9)	analog input 1
digital pin 4	(PCINT20/XCK/T0) PD4	6	23 PC0 (ADC0/PCINT8)	analog input 0
VCC	VCC	7	22 GND	GND
GND	GND	8	21 AREF	analog reference
crystal	(PCINT6/XTAL1/TOSC1) PB6	9	20 AVCC	VCC
crystal	(PCINT7/XTAL2/TOSC2) PB7	10	19 PB5 (SCK/PCINT5)	digital pin 13
digital pin 5 (PWM)	(PCINT21/OC0B/T1) PD5	11	18 PB4 (MISO/PCINT4)	digital pin 12
digital pin 6 (PWM)	(PCINT22/OC0A/AIN0) PD6	12	17 PB3 (MOSI/OC2A/PCINT3)	digital pin 11(PWM)
digital pin 7	(PCINT23/AIN1) PD7	13	16 PB2 (SS/OC1B/PCINT2)	digital pin 10 (PWM)
digital pin 8	(PCINT0/CLKO/ICP1) PB0	14	15 PB1 (OC1A/PCINT1)	digital pin 9 (PWM)

Digital Pins 11,12 & 13 are used by the ICSP header for MOSI, MISO, SCK connections (Atmega168 pins 17,18 & 19). Avoid low-impedance loads on these pins when using the ICSP header.

- จงเขียนโปรแกรมแสดง 0-9 โดยเว้นช่วงละ 1 วินาที



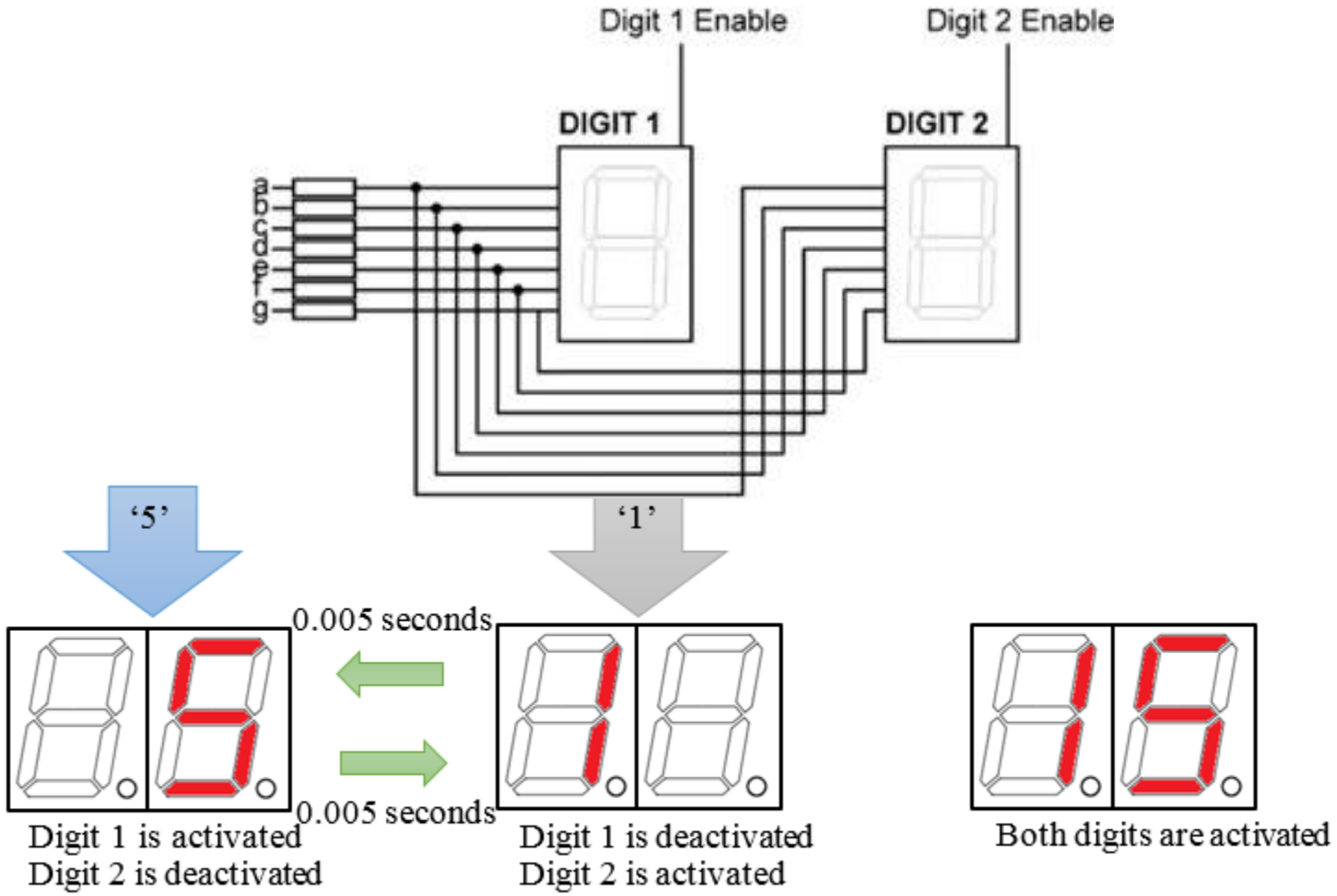


การจัดขาของ 7-segment

แบบฝึกหัดที่ 1.3 โปรแกรมควบคุม 7-segment

จงเขียนโปรแกรมแสดงเลขทีละ 2 หลักท้ายของรหัสนักศึกษา

เช่นรหัส 68070012 ก็แสดง 68 07 00 12 ช่วงละ 1 วินาที




THE END